

Metodología de desarrollo seguro de software basada en SAMM para el cumplimiento del estándar PCI-DSS en una empresa de pago electrónico de cuentas

Manuel Cortés Cortés
manuel.cortesc@sansano.usm.cl

Resumen: El mercado de las transacciones electrónicas de pago se encuentra en expansión y cada día se suman más empresas dispuestas a prestar estos servicios, aumentando la competencia de manera considerable. Por esta razón, las empresas dedicadas al negocio del procesamiento de pagos electrónicos deben estar preparadas y mantener siempre un alto nivel de calidad de servicio.

Uno de los aspectos importantes dentro de los requisitos que deben cumplir las empresas insertas en la industria de pago electrónico es lo relacionado con la seguridad de los datos y la prevención de fraudes. En este sentido existe la normativa internacional de seguridad PCI-DSS que se aplica a todas las entidades que almacenan, procesan o transmiten datos de titulares de tarjetas de pago. Esta normativa establece un nivel básico de protección para los consumidores y ayuda a reducir el fraude y las filtraciones de datos dentro de todo el ecosistema de pagos.

Para lograr una mejora en el cumplimiento de la normativa PCI-DSS existen diversas alternativas. Una de ellas es la aplicación de un modelo de madurez para el aseguramiento del software. Este trabajo propone la utilización de elementos del modelo SAMM de OWASP para intervenir el proceso actual de desarrollo de software y mejorar el nivel de cumplimiento de la normativa PCI-DSS en una empresa dedicada al procesamiento de pagos con tarjeta.

Esta intervención consistió en elaborar y probar una nueva metodología de desarrollo de software, basada en SAMM, para el desarrollo de software con mejores niveles de cumplimiento de las normas PCI-DSS. Para validar la propuesta se aplicó esta metodología a un proyecto de software sencillo. Además de la creación de una encuesta para medir el grado de aceptación y adaptabilidad de un equipo de desarrollo ágil a esta nueva metodología. Los resultados obtenidos indican que la nueva metodología contribuye a mejorar la seguridad de los productos de software.

Palabras Clave: Normas PCI-DSS, Modelos de Madurez, Modelo SAMM de OWASP, Ciclo de Desarrollo de Software Seguro.

1. Introducción

1.1 Contexto

Multicaja es una empresa del sector financiero dedicada al procesamiento de pagos con tarjeta dentro del mercado nacional y que, por la naturaleza de su negocio, debe procurar mantener constantemente un alto nivel de cumplimiento de los requisitos de seguridad establecidos por la industria. Más aún, si la empresa pretende expandir sus operaciones a otras regiones de América Latina.

Actualmente, la empresa cuenta con áreas de ingeniería de procesos, desarrollo y *testing* que trabajan de manera conjunta para la ejecución de proyectos de desarrollo de soluciones que soportan el negocio. Estas soluciones tienen relación, por ejemplo, con la gestión de inventarios de dispositivos, listas de precios, cuentas corrientes, facturación a clientes, entre otras. También se desarrollan aplicaciones para soportar, complementar y controlar las operaciones de los dispositivos POS¹ que, en estos momentos, representan una parte muy importante de los ingresos económicos de la empresa.

En el futuro inmediato, la empresa pretende expandir sus operaciones por medio de la creación de aplicaciones de pago para dispositivos móviles (*tablets* y *smartphones*), además de asociarse a nuevas marcas de tarjetas de

¹ Un POS (Point of Sale) es una agrupación de diferentes software y hardware que, al combinarse, permiten a las empresas procesar sus transacciones de cara al cliente. La configuración depende de los métodos de pago que una empresa acepte. Cada empresa utilizará su POS de acuerdo con sus necesidades comerciales.

pago; en consecuencia, deberá enfrentar desafíos importantes para los cuales se está preparando.

El objetivo de este trabajo es mejorar el nivel de cumplimiento de las normas de seguridad en la industria de las tarjetas de pago (Normas PCI-DSS) al interior de la organización, específicamente en el área de desarrollo de software, utilizando para ello el modelo SAAM de OWASP. Además, se busca establecer las bases de un proceso de mejora continua que permita a la empresa alcanzar mayores niveles de madurez en la seguridad de software e información, y evolucionar positivamente a medida que las normas y estándares aumentan sus niveles de exigencia.

1.1.1 Normativas de la CMF y ley de protección de datos.

En Chile existe una Ley Fintech que promueve la competencia e inclusión financiera a través de la innovación y tecnología en la prestación de servicios financieros. Y es regulada por la Comisión para el Mercado Financiero (CMF).

En ella se integraron temas relativos a la ciberseguridad a fin de que las organizaciones cumplan con los estándares mínimos de seguridad de la información y políticas de gestión de riesgos que la CMF establece. Todo esto para resguardar la privacidad de los clientes, además de la integridad, disponibilidad y confidencialidad de los datos involucrados en cada transacción.

Dentro de esta Ley la CMF tiene su Capítulo 20-10 Gestión De Seguridad De La Información Y Ciberseguridad: una regulación que deben cumplir las empresas financieras para proteger su información.

Por otro lado, la nueva Ley de Protección de Datos actualiza la legislación vigente y eleva el estándar de protección a los derechos de las personas. Con esto, el estándar chileno se homologa al establecido por el Reglamento General de Protección de Datos de la Unión Europea, siendo la referencia internacional para la protección de los derechos de las personas y sus datos.

La nueva ley establece y regula detalladamente los derechos de los titulares de los datos personales, y los medios para que éstos hagan valer estas garantías ante los responsables de datos.

Esta regulación se aplicará a:

- Quienes realicen tratamiento de datos personales en el territorio nacional
- Quienes realicen tratamiento de datos a nombre de un mandatario que se encuentre en el territorio nacional
- Quienes realicen tratamiento de datos destinado a ofrecer bienes o servicios a personas que se encuentren en el país.

En estos dos últimos casos, esta regulación se aplicará incluso a quienes traten los datos personales sin encontrarse presentes en el territorio nacional.

La ley crea una institucionalidad para la protección de los datos personales denominada Agencia de Protección de Datos Personales, la que tendrá como objetivo fiscalizar el cumplimiento de las disposiciones de esta ley y aplicar sanciones.

Para esto, la ley establece un catálogo de conductas e infracciones: leves, graves y gravísimas, las que se sancionarán con multas que se encuentran entre las más altas de nuestro ordenamiento jurídico. Y entrega un plazo de 24 meses a partir de su publicación a todos los responsables de datos para que se adapten a este nuevo régimen, entre los que se encuentra la organización bajo estudio.

Esta reforma, a su vez, permitirá a Chile ser declarado por la Comisión Europea como país con un nivel adecuado de protección de datos personales, lo que facilitará la transferencia internacional de datos entre nuestro país y la Unión Europea, importantísimo socio comercial de Chile.

Así, a la vez que vela por la protección de los derechos de las personas bajo estándares internacionales, esta ley regula y promueve la economía digital en nuestro país.

Estas leyes serán reguladas y supervisadas por organismos gubernamentales, ampliando el espectro de datos y complementándose con las normativas PCI-DSS, ya que, al igual de ésta, hacen énfasis en la seguridad de los procesos y la protección de datos, con lo cual las empresas deberán ajustar sus procesos de desarrollo a estas nuevas regulaciones. Además, el uso de modelos de madurez para cumplir con estas regulaciones son herramientas de gran utilidad.

1.2 Definición del problema y motivación

Debido a que la empresa opera dentro de la industria de los medios electrónicos de pago, la cual se encuentra regida por estándares y normas de seguridad, es importante que las áreas estén actualizadas respecto de los procedimientos que se deben seguir para garantizar la seguridad de los datos y los procesos, ya que, como toda empresa del sector, se encuentra expuesta a posibles fraudes y sustracción de datos sensibles.

La norma más relevante que debe cumplir la empresa es la que tiene relación con las operaciones con tarjetas de pago. Esta corresponde al Estándar de Seguridad de Datos para la Industria de Tarjeta de Pago, más conocido como PCI-DSS², cuyo incumplimiento puede ocasionar perjuicios económicos e incluso la pérdida del negocio por parte de la empresa. Por esta razón es fundamental que la organización mantenga y, de ser posible, mejore los niveles de cumplimiento relacionados a esta norma.

Para mantener el nivel de cumplimiento de los estándares es necesario que la organización cuente con una herramienta que le permita evaluar, tomar decisiones y actuar para realizar acciones correctivas de sus procesos de desarrollo de software. Esto debe entregar como resultado productos más seguros y mantenibles en el tiempo.

En estos momentos no existe una metodología definida que establezca etapas dentro del ciclo de desarrollo de software que permita la construcción de productos que cumplan adecuadamente con los requerimientos de seguridad y cumplimiento de normativas externas. Esto se hace, por lo general, de manera intuitiva por parte de los equipos de desarrollo.

Por otro lado, se requiere que los productos de software cuenten con documentación suficiente para apoyar futuras mantenciones, mejoras y resolución de incidencias, reduciendo el esfuerzo necesario para estas actividades. Lo anterior implica la necesidad de la organización de contar con una metodología clara y definida que permita a los equipos de trabajo desarrollar actividades por etapas que entreguen como resultado productos de software que cumplan con los estándares de seguridad exigidos, sean mantenibles y que dispongan de la documentación necesaria y suficiente.

1.3 Alternativas de solución existentes

Existen alternativas que permiten a las organizaciones alcanzar mayores niveles de madurez en cuanto a la postura de seguridad y mejorar los niveles de cumplimiento de los requisitos de las normas PCI- DSS.

1.3.1 Asesorías externas especializadas

Una alternativa para lograr los objetivos de cumplimiento de las normas PCI-DSS y el mejoramiento de los niveles de seguridad es recurrir a empresas externas especializadas en estos temas. Estas empresas cuentan con certificaciones verificables que las habilitan para entregar servicios de consultoría en ciberseguridad y acompañar a las empresas del sector financiero para el cumplimiento de normas y estándares.

1.3.2 Software de cumplimiento

El mercado ofrece software que sirve de guía a las empresas para el cumplimiento de diversas normativas, que definen normas y estándares que la empresa debe cumplir y, en el momento oportuno, se envía la documentación y las tareas correspondientes para asegurar que la organización cumpla con estas normativas.

Algunos ejemplos de estos productos de software son:

² Por su sigla del inglés: Payment Card Industry Data Security Standard (PCI-DSS).

- **EventLog Analyzer**³. Este software de administración y análisis de registros de ManageEngine, ayuda a establecer el cumplimiento de PCI-DSS con su interfaz basada en web de “*simple diseño y funciones sofisticadas*” e informes específicos de cumplimiento.
- **GlobalSuite**⁴. Este software permite realizar análisis diferenciales sobre normativa PCI, definición de niveles de madurez para cada requisito, visualización del resultado a través de diferentes gráficos, comparación de resultados de diferentes análisis y exportación de la información para reportes.
- **Netwrix**⁵. Este software de cumplimiento de PCI-DSS permite controlar cambios críticos, configuraciones y eventos de acceso. También proporciona inteligencia de seguridad que ayuda a identificar agujeros de seguridad, detectar anomalías en el comportamiento de los usuarios e investigar patrones de amenazas a tiempo para evitar daños reales. Finalmente, incluye informes de cumplimiento para requisitos específicos de muchos estándares de cumplimiento comunes, incluido PCI-DSS.

Dentro de las características de estos productos/herramientas de software, se encuentran funcionalidades que ayudan a las organizaciones a:

- Descubrir e informar sobre los datos de las tarjetas de pago en los entornos de almacenamiento.
- Auditar cómo se protegen, procesan y transmiten datos sensibles.
- Monitorear la integridad de los archivos en el entorno de los datos de las tarjetas.
- Proporcionar una información detallada de los permisos de seguridad y del almacenamiento de archivos.
- Proteger los archivos sensibles de las pérdidas de datos accidentales y maliciosas.

1.3.3 Adquirir software certificado para el procesamiento de pagos

El mercado ofrece soluciones de software certificadas PCI-DSS que pueden ser adquiridas por las empresas que se dedican al procesamiento de pagos con tarjeta.

1.3.4 Utilizar una metodología de desarrollo seguro

Existen metodologías de desarrollo seguro implementadas y aplicadas por grandes empresas de desarrollo de software. Entre éstas se destacan:

- **Microsoft Trustworthy Computing SDL [1]**. Este ciclo de vida de desarrollo de seguridad de Microsoft es un proceso utilizado y propuesto por Microsoft para reducir los costos de mantenimiento y aumentar la confiabilidad del software en relación con los errores relacionados con la seguridad.
- **Oracle Software Security Assurance Process [2]**. Esta es la metodología de Oracle para crear seguridad en el diseño, el desarrollo, las pruebas y el mantenimiento de los productos de software, abarcando cada fase del ciclo de vida del desarrollo del producto.

1.3.5 Aplicación de modelos de madurez

Existen modelos de madurez para el desarrollo de software seguro que pueden ser utilizados para mejorar la postura de seguridad de las organizaciones. Estos modelos definen actividades que, al ser incluidas dentro del proceso de desarrollo, entregan como resultado un ciclo de desarrollo seguro y, por lo tanto, productos de software que cumplen de mejor manera con los estándares y normas de seguridad exigidas por la industria.

Los modelos de madurez más reconocidos son:

- Building Security In Maturity Model (BSIMM) [3]
- SAMM de OWASP [4]

1.3.6 Crear una metodología ad-hoc en base a algún modelo de madurez

Es posible que la organización desarrolle una metodología propia que se ajuste a las necesidades de la empresa y a los recursos disponibles, utilizando como base uno de los modelos mencionados anteriormente. Esto requiere que el modelo seleccionado se encuentre documentado, sea flexible y fácil de adaptar a la organización.

³ GlobalSuite Solutions, url : <https://www.globalsuitesolutions.com/pci-dss/>

⁴ GlobalSuite Solutions, url : <https://www.globalsuitesolutions.com/cl/>

⁵ Netwrix, url : https://www.netwrix.com/PCI_Compliance.html

1.4 Enfoque de solución propuesto

La solución propuesta consiste en la definición y aplicación de una metodología de desarrollo seguro basada en el modelo de madurez para el aseguramiento de software (SAMM) de OWASP. Esto requiere de la intervención del actual proceso de desarrollo de software, incorporando y adaptando a éste algunas actividades y elementos propios del modelo. Con ello se pretende mejorar los actuales niveles de seguridad de datos y aplicaciones que son parte de los activos informáticos de la empresa. Además, se tiene como finalidad, mejorar los niveles de cumplimiento de las normas externas a las que está sujeta la empresa (PCI-DSS).

Se ha seleccionado dicho modelo de madurez debido a sus características:

- Es un marco de trabajo abierto, de libre uso y apoyado por una comunidad activa.
- Es flexible y se adapta bien a las necesidades de las organizaciones.
- Se encuentra bien documentado.
- Existen herramientas de evaluación, *testing* y análisis de código que son de uso libre.

Además, se requiere que la organización cuente con una metodología de trabajo que incluya una herramienta para evaluar cuál es el nivel de madurez en que se encuentra respecto de la seguridad de sus datos y sistemas, para que, de este modo, establezca planes de mejora continua que le permitan alcanzar niveles más altos.

1.5 Hipótesis y metodología de validación

1.5.1 Hipótesis

La aplicación de una metodología de desarrollo seguro de software basada en SAMM de OWASP en un proceso de desarrollo de software basado en agilidad, permitirá:

1. Una intervención mínima.
2. Mejorar el cumplimiento de estándares PCI DSS.
3. Con un costo-efectivo menor (considerando gestión de riesgo y resolución de incidentes).

1.5.2 Metodología de validación

Para validar esta propuesta se ejecutarán las siguientes acciones:

- Análisis de la metodología actual: Se hará un análisis de las actividades que forman parte del actual proceso de desarrollo de software al interior de la organización. Esto tiene como objetivo establecer qué actividades del modelo de madurez SAMM de OWASP serán agregadas para mejorar la seguridad de los productos de software desarrollados al interior de la empresa.
- Diagnóstico Inicial: Se realizará un levantamiento de la situación actual de la organización respecto a las posturas de seguridad en el proceso de desarrollo de software. Esta evaluación se realizará mediante la utilización de cuestionarios y herramientas de medición definidas para este propósito.
- Aplicación de la Metodología: Una vez realizado el análisis descrito en el punto anterior, se hará una intervención en el proceso de desarrollo para un proyecto de software, introduciendo la nueva metodología definida en base al modelo de madurez y desarrollando las actividades que permitan mejorar los niveles de seguridad.
- Proceso de medición: Finalizadas las etapas anteriores, se realizará un nuevo diagnóstico con el fin de verificar si la aplicación de la metodología dio como resultado, a juicio del equipo de trabajo, una mejora en la seguridad del software desarrollado con respecto a la situación inicial.

1.6 Estructura del informe

Este trabajo está compuesto por cinco capítulos. En el capítulo 2 como Marco Teórico y Estado de Arte, se presentan antecedentes sobre las normas PCI-DSS, modelos de madurez para el desarrollo de software seguro, una comparación entre modelos de madurez, metodología Agile y modelos de madurez y algunos trabajos relacionados. A continuación, en el capítulo 3, se presenta la Propuesta de Solución y Caso de Estudio, describiendo la metodología actual de desarrollo de software al interior de la organización bajo estudio, una autoevaluación de los niveles de seguridad, utilizando el modelo SAMM de OWASP y la metodología propuesta para mejorar el cumplimiento de las normas PCI-DSS. En el capítulo 4, la metodología propuesta se valida comparando ambos procesos de desarrollo (actual y propuesto). Para ello, se ejecuta el nuevo proceso en un proyecto de corta duración y se recogen los comentarios, apreciaciones y críticas del equipo de trabajo.

Finalmente, se concluye con un análisis del trabajo realizado y la validación de la metodología propuesta.

Se incluyen anexos con descripciones detalladas y referencias a herramientas que se pueden utilizar para apoyar las actividades consideradas en la metodología propuesta.

2. Marco Teórico y Estado de Arte

2.1 Normas de seguridad de datos PCI DSS Versión 3.2.1

Debido al incremento de compras de bienes y servicios a través de Internet, las empresas que intervienen en estas operaciones deben procesar, almacenar y gestionar la información de los titulares de las tarjetas de pago de manera segura, esto con el fin de evitar fraudes y el acceso no autorizado a esta información por parte de *hackers*.

Para impedir estos inconvenientes, un grupo de empresas emisoras de tarjetas de crédito crearon el PCI Security Standard Council⁶, un foro mundial en el que se establecen mejoras de almacenamiento, difusión y cumplimiento de normas de seguridad de datos. A estas normas se les denomina Payment Card Industry Data Security Standard (PCI-DSS) [5].

Las normas PCI-DSS están creadas para guiar a las organizaciones de manera proactiva y eficiente en la protección de los datos de sus clientes. Estas normas presentan los requisitos, políticas, procedimientos y arquitectura de redes, entre otras definiciones, para lograr una adecuada protección de los datos, gestionar las vulnerabilidades e implementar medidas sólidas de seguridad y otras que favorecen la mejora de las medidas de protección a nivel mundial.

Las normas PCI-DSS se aplican a todas las entidades que participan en el procesamiento de las tarjetas de pago, entre las que se incluyen: comerciantes, pasarelas de pagos, procesadores, entidades emisoras, etc. Las normas PCI-DSS se aplican a todas las entidades que almacenan, procesan o transmiten datos del titular de la tarjeta (CHD)⁷ y/o datos confidenciales de autenticación (SAD)⁸ y cuyo incumplimiento está sujeto a multas o penalizaciones.

A continuación, en la Tabla 1 se presenta una descripción de alto nivel de las normas PCI DSS.

Tabla 1. Normas de seguridad de datos PCI - DSS: descripción general de alto nivel [5]

Categoría		Requerimiento
Desarrolle y mantenga redes y sistemas seguros.	1	Instalar y mantener una configuración de <i>firewall</i> para proteger los datos del titular de la tarjeta.
	2	No usar los valores predeterminados suministrados por el proveedor para las contraseñas del sistema y otros parámetros de seguridad.
Proteger los datos del titular de la tarjeta.	3	Proteger los datos del titular de la tarjeta que fueron almacenados.
	4	Cifrar la transmisión de los datos del titular de la tarjeta en las redes públicas abiertas.
Mantener un programa de administración de vulnerabilidad.	5	Proteger todos los sistemas contra <i>malware</i> y actualizar los programas o software antivirus regularmente.
	6	Desarrollar y mantener sistemas y aplicaciones seguros.
Implementar medidas sólidas de control de acceso.	7	Restringir el acceso a los datos del titular de la tarjeta según la necesidad de saber que tenga la empresa.
	8	Identificar y autenticar el acceso a los componentes del sistema.
Supervisar y evaluar las redes con regularidad.	9	Restringir el acceso físico a los datos del titular de la tarjeta.
	10	Rastrear y supervisar todos los accesos a los recursos de red y a los datos del titular de la tarjeta.
Mantener una política de seguridad de la información.	11	Probar periódicamente los sistemas y procesos de seguridad.
	12	Mantener una política que aborde la seguridad de la información para todo el personal.

2.2 Modelos de madurez para el ciclo de desarrollo seguro de software

Un modelo de madurez es un instrumento que ayuda a evaluar la eficacia de los procesos o prácticas de personas, grupos u organizaciones y a determinar qué capacidades deben ser adquiridas o evolucionadas para

⁶ PCI Security Standard Council fue formado originalmente por American Express, Discover Financial Services, JCB International, MasterCard y Visa Inc., el 7 de septiembre de 2006.

⁷ CHD = Acrónimo de "Cardholder Data"; consta del número de cuenta principal, el nombre del titular de la tarjeta, la fecha de vencimiento de la tarjeta y, en ocasiones, el código de servicio.

⁸ SAD = Acrónimo de "Sensitive Authentication Data", incluye la información de la pista magnética, el número confidencial o clave (PIN), así como el valor de autorización de tarjeta no presente al que se denomina como CVV2.

mejorar su desempeño.

Los modelos de madurez contienen los elementos esenciales de procesos eficaces, para una o más disciplinas, y proporcionan una hoja de ruta para la mejora evolutiva, desde procesos ad-hoc e inmaduros, hasta alcanzar procesos disciplinados y maduros de mayor calidad y eficacia [6].

Es importante que cada organización que haga uso de software, ya sea de terceros o propio, tenga un marco de referencia para comprender cuál es el nivel de seguridad con el que cuenta. Es en este punto en que los modelos de madurez de desarrollo de software seguro ayudan a establecer la situación actual de la organización respecto del tema de la seguridad y cuál es el nivel de madurez que se debe alcanzar para considerar que se tiene un buen control de la situación.

Además, es importante realizar una revisión de los requisitos a medida que éstos evolucionan en el modelo de madurez, ya que éstos cambian acorde a la evolución de las amenazas de seguridad a las que se enfrentan las organizaciones. Por esta razón, el uso de este tipo de modelos se considera como base para la creación de un proceso de mejora continua.

En relación al desarrollo de software seguro, existen dos modelos de madurez específicos para este propósito, a saber: el Building Security In Maturity Model (BSIMM) de Synopsys y el Software Assurance Maturity Model (SAMM) de OWASP.

2.2.1 Modelo de madurez BSIMM Versión 5.0

El modelo BSIMM [3] es el resultado de un estudio sobre las iniciativas de seguridad en el desarrollo de software que se llevan a cabo, de manera frecuente, en una serie de organizaciones, principalmente empresas de Estados Unidos. Específicamente, el modelo se construye a partir de los datos observados en grandes empresas, recopilando las diversas iniciativas de seguridad del software, normalizando la información para describirla de manera uniforme, sin que influya la utilización de diferentes metodologías y terminologías. Esta estandarización permite disponer de un *framework* cuantificable y utilizable por cualquier organización, permitiendo comparar las iniciativas de seguridad a pesar de que operen en diversas escalas, se lleven a cabo en diferentes partes del organigrama o creen diferentes artefactos de trabajo.

El modelo BSIMM no es un modelo de madurez tradicional, en el que un conjunto de actividades se repite a lo largo de los diversos niveles de madurez. En el caso de BSIMM, las actividades son únicas con niveles de actividad utilizados únicamente para mostrar la frecuencia relativa con la que las actividades se observan en las diferentes organizaciones. Así, las actividades que se observan con mayor frecuencia en el conjunto de organizaciones analizado se designan como nivel 1, las que se observan con menor frecuencia se designan como nivel 2 y las que se observan con muy poca frecuencia se designan como nivel 3. Es decir, se asume que hay un conjunto de prácticas que son realizadas por la mayoría de las organizaciones (nivel inicial de madurez). Las actividades de menor frecuencia, más avanzadas o complejas se asocian con niveles de madurez más altos.

BSIMM no establece ninguna obligación de ejecutar ciertas prácticas observadas en otras organizaciones. Simplemente informa de las prácticas de seguridad que se realizan en las organizaciones evaluadas. Cada organización debe determinar, en función de sus objetivos de seguridad, qué prácticas va a ejecutar. El conocimiento de lo que la mayoría, una parte u un conjunto reducido de organizaciones hace en materia de seguridad del software sirve como base para elaborar la estrategia de seguridad de la organización.

El uso más relevante de BSIMM es servir como una referencia para determinar dónde se sitúa actualmente una organización en relación con otras, pues permite comparar y contrastar las iniciativas de seguridad de una organización respecto a lo que hacen las organizaciones analizadas. Esto proporcionará una idea del estado de seguridad del software de la organización, en relación al grado de madurez medio del resto de organizaciones.

Adicionalmente, el modelo puede utilizarse para establecer una hoja de ruta del programa de seguridad en el software. Para ello se compara la organización con el resto, se evalúa la distancia que las separa y se prepara el plan de acción oportuno para reducir o eliminar la diferencia. De igual forma, el modelo se puede emplear para evaluar o medir la evolución en el tiempo de los esfuerzos invertidos en el programa de seguridad.

El modelo BSIMM se organiza en 12 actividades de seguridad agrupadas en 4 dominios:

- **Gobernanza:** Es el primer dominio y en él son muy importantes la estrategia, la planificación, la asignación de roles, la identificación de metas de seguridad del software y la determinación de presupuestos. En este dominio también es fundamental la capacitación del personal. Las tres prácticas asociadas a este dominio son: Estrategia y métricas, Cumplimiento y política, y Capacitación.
- **Inteligencia:** Permite estar atento a cualquier ataque en la seguridad del software, utilizando la herramienta Modelos de Ataque. Con esta se captura la información, se modelan las amenazas, se desarrollan los casos de uso, se realiza la clasificación de datos y se definen los patrones de ataque específicos. También se maneja la práctica Características de Seguridad y Diseño, con la cual se manejan patrones de seguridad más importantes, y la práctica de Normas y Requisitos, donde se plasma toda la reglamentación que se debe cumplir en materia normativa y legal.
- **Puntos de Contacto con el SSDL (Secure Software Development Lifecycle):** En este dominio se pueden encontrar dos prácticas de seguridad que son consideradas las más importantes, como son el Análisis de la Arquitectura y la Revisión de Código. Con éstas se puede realizar un plan de evaluación y correcciones para la organización, de forma que se realicen las correcciones necesarias y las actualizaciones precisas.
- **Despliegue:** Son pruebas de penetración tercerizadas para obtener un punto de vista externo. Estas pruebas principalmente estudian los puntos más vulnerables de la configuración final y así mismo se retroalimentan directamente con la información obtenida sobre defectos y mitigación de los mismos.

La Tabla 2 presenta la estructura del modelo BSIMM, sus dominios y las 12 las actividades que lo forman.

Tabla 2. Estructura modelo BSIMM. Fuente: McGraw, Gary. Building Security in Maturity Model. Release 5.0 [3].

BSIMM			
Gobernanza	Inteligencia	Puntos de Contacto con el SSDL	Despliegue
Estrategia y Métricas	Modelos de Ataque	Análisis de la Arquitectura	Pruebas de Penetración
Cumplimiento y Política	Características de Seguridad y Diseño	Revisión de Código	Entorno del Software
Capacitación	Normas y Requisitos	Pruebas de Seguridad	Gestión de Configuración y Gestión de Vulnerabilidades

Cada dominio y cada práctica tienen una meta de alto nivel. El dominio de gobernanza busca mejorar la transparencia y cumplimiento del nivel de seguridad del modelo. El de inteligencia crea recursos propios para predecir ataques, crear patrones de seguridad y para la creación de normativas de seguridad propias. Los puntos de contacto incluyen las prácticas del SSDL con el objetivo de controlar la calidad del software desarrollado. Por último, el despliegue también tiene como meta el control de la calidad a través de pruebas de penetración y el manejo de incidentes.

2.2.2 Modelo de madurez SAMM de OWASP Versión 2.0

Para apoyar a las organizaciones a implementar estrategias de seguridad para el software que estén acorde a las necesidades que enfrentan, la fundación OWASP⁹ ha desarrollado un marco de trabajo abierto, conocido como Modelo de Madurez para el Aseguramiento del Software (SAMM) [4].

SAMM se ha elaborado con un planteamiento agnóstico en cuanto al paradigma de desarrollo que cada organización tenga establecido, aceptando metodologías de desarrollo en cascada, iterativas, agile, DevOps, etc. Por tanto, puede ser empleado por cualquier tipo de organización.

SAMM proporciona un medio para conocer en qué estado se encuentra una organización y determinar qué pasos dar para elevar su nivel de madurez al siguiente nivel. Los pasos a seguir consisten en realizar diferentes prácticas concretas durante el ciclo de vida del desarrollo de software que conducirán a mejorar el estado de seguridad de los productos finales. Estas prácticas se agrupan en funciones de negocio que se corresponden con categorías de actividades relacionadas con el proceso de desarrollo, y que cualquier organización que produzca software debe cumplir en mayor o menor medida. Las funciones de negocio son los elementos de

⁹ OWASP es un proyecto en una comunidad abierta dedicada a determinar y combatir las causas que hacen que el software sea inseguro, produciendo para este propósito recursos que están disponibles en forma gratuita.

más alto nivel en los que se fundamenta el modelo. A continuación, se presentan las cinco funciones de negocio definidas en SAMM versión 2.0:

- Gobierno
- Diseño
- Implementación
- Verificación
- Operación

Para cada una de estas funciones, SAMM define tres prácticas cuya finalidad es garantizar la seguridad de dicha función de negocio.

En cada práctica de seguridad, se definen a su vez tres niveles de madurez. Cada nivel persigue un objetivo más sofisticado que el anterior, con actividades más exigentes y métricas más estrictas.

Adicionalmente, para cada una de las prácticas SAMM define dos líneas de acción que agrupan varias actividades. Cada línea de acción persigue un objetivo que se puede alcanzar con cierto nivel de madurez. Las líneas de acción vinculan las diferentes actividades de la práctica con los niveles de madurez establecidos.

En la Figura 1 se muestra la estructura en que están organizados los elementos que forman el modelo:

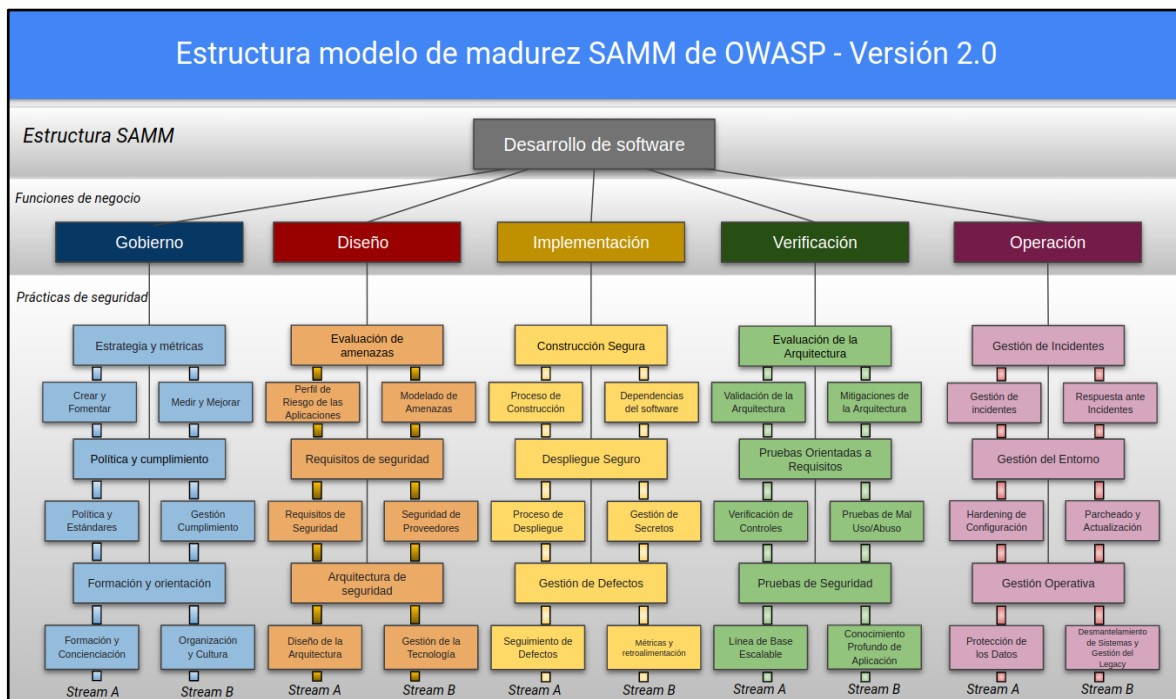


Figura 1. Modelo SAMM Versión 2.0. Fuente: "OWASP SAMM" [4]

Las características principales del modelo SAMM son las siguientes:

- Medible: niveles de madurez definidos en todas las prácticas de seguridad.
- Accionable: vías claras para mejorar los niveles de madurez.
- Versátil: agnóstico a la tecnología, los procesos y la organización.

Las indicaciones proporcionadas por el modelo de madurez ayudan a las organizaciones a:

- Evaluar las prácticas de seguridad de los diversos procesos llevados a cabo a lo largo del ciclo de vida de desarrollo de software y medir el grado de madurez de la organización en este aspecto.
- Evidenciar la ejecución de buenas prácticas en el desarrollo de software.
- Definir las prácticas y actividades que se llevarán a cabo para la mejora de la seguridad en el proceso de desarrollo de software.
- Desarrollar un programa incremental e iterativo de aseguramiento de la seguridad del software.
- Demostrar progresos concretos del programa de mejora de la seguridad en el desarrollo de software.

Una característica importante de este modelo es que está definido de manera flexible, lo que permite su

utilización por organizaciones de cualquier tamaño. Además, el modelo puede ser aplicado a la organización por completo, a una línea de negocio o a un proyecto particular.

El modelo SAAM está construido sobre una serie de principios, entre los que se encuentran:

- Ser consistente con los cambios de comportamiento de una organización a través del tiempo. Un programa exitoso de seguridad de software debería ser creado en pequeños ciclos que entreguen ganancias tangibles, de esta manera se puede apreciar los beneficios de su implementación en el corto plazo, logrando un mayor interés por la organización. Al mismo tiempo, se debe trabajar de manera incremental con el objetivo de alcanzar metas de largo plazo.
- No existe una receta única que funcione para todas las organizaciones. El modelo es flexible y permite que las organizaciones personalicen sus opciones de acuerdo al grado de tolerancia al riesgo que tienen y la metodología con la que desarrollan y usan el software.
- Los lineamientos relacionados a las actividades de seguridad deben ser específicos. El modelo establece que todos los pasos en la construcción y medición del programa de aseguramiento deben ser simples, bien definidos y medibles, proporcionando plantillas de planes de implementación para tipos comunes de organizaciones.

2.2.2.1 Cómo utilizar SAMM de OWASP

El primer paso para la utilización del modelo SAMM es evaluar la postura actual de la organización con respecto de la seguridad de sus activos de información y las actividades de seguridad existentes. Esto se realiza por medio de hojas de trabajo con un conjunto de preguntas que permiten a la organización realizar una autoevaluación de su situación actual. Con esto se logra establecer el nivel de las actividades de seguridad que ya se encuentran creadas. Esta revisión permite determinar el nivel de madurez en el cual se está desempeñando la organización. La profundidad hasta la cual se debe revisar el desempeño puede variar dependiendo de los objetivos de la revisión, pero en general, hay dos estilos recomendados:

- **Ligero:** Las hojas de trabajo de cada práctica son evaluadas y las calificaciones son asignadas en base a las respuestas. Este tipo de revisión es suficiente para una organización que desea tener rápidamente una idea de cuál es su posición respecto al modelo.
- **Detallado:** Después de completar la revisión de las hojas de trabajo, se realiza un trabajo de auditoría adicional para verificar que las actividades recomendadas por cada práctica existan y funcionen.

Como ya se mencionó, se debe dar respuesta, por parte de la organización, a un conjunto de preguntas relacionadas con las funciones de negocio y prácticas de seguridad definidas. Luego de tener las respuestas se debe evaluar la postura de la organización y clasificar su nivel de acuerdo a la cantidad de respuestas afirmativas y la escala de niveles de madurez definida en el modelo.

2.2.2.2 Niveles de Madurez

Cada una de las prácticas de seguridad tiene tres niveles de madurez bien definidos y un nivel inicial (cero) implícito. Los detalles de cada nivel difieren entre las prácticas, pero generalmente representan:

- 0 - Punto de inicio implícito, las actividades en la práctica no se han realizado
- 1 - Entendimiento inicial y provisión ad hoc de la práctica de seguridad
- 2 - Incremento en la eficiencia y/o efectividad de la práctica de seguridad
- 3 - Dominio amplio de la práctica de seguridad

Los programas de aseguramiento ya existentes no siempre consisten en actividades que se ajustan exactamente a los límites de los niveles de madurez; por ejemplo, una organización que verifica si está en nivel 1 para una práctica en particular puede también tener actividades adicionales, pero que no cumplen completamente con el nivel 2. Para esos casos, la calificación debe ser anotada con un símbolo de “+” para indicar que hay medidas implementadas más allá de lo requerido por el nivel.

Al tener una visión clara de la situación actual, se debe seleccionar de manera prioritaria cuáles de los actuales niveles de madurez deben ser mejorados, desarrollando las actividades establecidas en el modelo para ese propósito o complementando las ya existentes.

2.2.3 Comparación entre los modelos BSIMM y SAMM

Primero es necesario entender la principal diferencia entre los dos modelos: BSIMM es un modelo descriptivo, mientras que SAMM es un modelo prescriptivo. Como se ha comentado, BSIMM proporciona información sobre qué prácticas de seguridad está aplicando la industria en sus procesos de desarrollo, mediante el análisis de las iniciativas de seguridad de un grupo amplio de organizaciones. Es decir, no indica lo que se debe hacer, sino que presenta lo que la industria hace con el fin de proporcionar orientación a las organizaciones que deseen poner en marcha un programa de seguridad en el software.

Por el contrario, SAMM sí define un conjunto de prácticas que se han de ejecutar para que una organización establezca un programa de seguridad que la ayude a tener un proceso de desarrollo de software más seguro. Las prácticas prescritas por SAMM son el resultado de un proceso de evaluación por parte de los equipos del proyecto OWASP que, sobre la base de su conocimiento y experiencia, determinan la eficacia de las actividades sugeridas en el modelo. Su objetivo es ayudar a las organizaciones a definir las actividades relacionadas con la seguridad para implementar un programa de seguridad por iteraciones y evaluar los progresos realizados.

En relación a los niveles de madurez, ambos modelos establecen 3 niveles de madurez, pero se conciben de forma diferente. Como ya se ha mencionado, BSIMM basa su modelo de madurez según la frecuencia relativa en la que se observan diferentes prácticas en las organizaciones. Por su parte, SAMM establece un enfoque más tradicional, en el que las diferentes prácticas se repiten a lo largo de los niveles de madurez, con un incremento de la exigencia a medida que se avanza de nivel.

Otra de las diferencias importantes es que BSIMM es un modelo propietario de la compañía Synopsys, por lo que para realizar una evaluación es preciso contactarse con dicha organización con el fin de presupuestar y planificar el proceso. Por su parte, SAMM es un modelo abierto y puede ser utilizado sin limitaciones. Además, OWASP ofrece diversas herramientas para realizar una autoevaluación e incluso realizar seguimiento de la evolución del programa de seguridad en el tiempo. Por supuesto, también se puede contratar la evaluación a empresas auditoras del modelo.

La frecuencia de actualización de los modelos también presenta diferencias. BSIMM lleva a cabo un análisis anual de las empresas que participan en el desarrollo del modelo, evaluando sus prácticas de seguridad. Por tanto, el modelo se actualiza cada año. En cambio, SAMM se actualiza cada 2 o 3 años.

En lo que respecta a la comparación del grado de madurez de una organización con respecto a otras, BSIMM sí permite contrastar los resultados para conocer el estado de seguridad de una organización en relación con el resto de las empresas que participan en este *framework*. De hecho, tal y como ellos mismos reconocen, este es uno de los principales objetivos de BSIMM. Esto es algo que por el momento no es posible realizar con SAMM.

A continuación, en la Tabla 3, se presenta un resumen comparativo entre ambos modelos.

Tabla 3. Comparación entre BSIMM v/s SAMM

Comparando	BSIMM	SAMM
Tipo de modelo	Descriptivo	Prescriptivo
Actividades	12 actividades observadas y relacionadas por área de práctica.	2 flujos de actividades por práctica de seguridad que se complementan entre sí.
Niveles de madurez	BSIMM comprende un conjunto de actividades únicas, con niveles de actividad utilizados solo para distinguir la frecuencia relativa con la que se observan las actividades en las organizaciones. Las actividades observadas con frecuencia se designan como nivel 1, las actividades observadas con menos frecuencia se designan como nivel 2 y las actividades observadas con poca frecuencia se designan como nivel 3.	En SAMM, cada una de las prácticas de seguridad tiene tres niveles de madurez definidos y un punto de partida implícito en cero. Los detalles para cada nivel difieren entre las prácticas, pero generalmente representan: 0: Punto de partida implícito que representa las actividades en la práctica sin cumplir. 1: Comprensión inicial y provisión ad-hoc de la práctica de seguridad. 2: Incremento de la eficiencia y/o eficacia de la práctica de seguridad. 3: Dominio integral de la práctica de seguridad a escala.

Comparando	BSIMM	SAMM
Comparación de resultados	BSIMM 11 cuenta con la colaboración de numerosas organizaciones que son entrevistadas y contribuyen a la recopilación del conjunto de datos de comparación.	No existen datos para comparar entre diferentes organizaciones.
Frecuencia de actualizaciones	Anual.	Cada 2-3 años.
Evaluación	BSIMM es propiedad de Synopsys. Para una evaluación, debe comunicarse con ellos para conocer el costo y la logística.	SAMM es un modelo abierto y puede ser autoevaluado o realizado por varias organizaciones e individuos diferentes.

Con respecto a su organización y estructura, los modelos presentan bastantes semejanzas, ya que en ambos se definen categorías de alto nivel (dominios o funciones según el modelo), cada una de las cuales contienen un conjunto de prácticas (3 por dominio o función), que a su vez incluyen un conjunto de actividades.

2.2.4 Selección del modelo de madurez

Al analizar y comparar ambos modelos se establece que SAMM de OWASP es más apropiado para ser aplicado como base de la solución propuesta al problema presentado en este trabajo.

Esta decisión se justifica en el hecho de que SAMM presenta las siguientes características:

- Es un modelo prescriptivo que puede ser aplicado a organizaciones de cualquier tamaño, equipos de desarrollo, e incluso a proyectos específicos.
- Es un modelo incremental que permite avanzar en los niveles de madurez a medida que se incorporan nuevas actividades.
- Es un modelo abierto que se puede ser aplicado sin realizar ningún pago a sus creadores.
- Es apoyado por una gran comunidad que aporta con su experiencia a su perfeccionamiento.
- Existen herramientas de apoyo a las actividades y que se encuentran validadas por la comunidad OWASP.
- Proporciona herramientas para realizar autoevaluaciones.

2.3 Metodología Agile y Modelo de Madurez SAMM

Puede parecer que un modelo de madurez entra en conflicto con la metodología Agile, debido a que Agile pone a las personas por encima del proceso y al software por encima de la documentación. Sin embargo, Agile no descalifica el proceso o la documentación, lo que realmente pretende esta metodología es minimizar esto donde sea posible. Por esta razón, un modelo de madurez es de gran ayuda para apoyar la metodología y mejorar la forma en que trabaja.

El modelo de madurez ayuda a integrar dentro de la metodología Agile aspectos como [7]:

2.3.1 Autonomía del equipo

En un entorno ágil la autonomía del equipo es importante, por lo cual, el programa de madurez debe comprender que diferentes equipos tienen diferentes niveles de madurez y diferentes formas de trabajar, incluida la seguridad. Al mismo tiempo, es recomendable dejar que los equipos aprendan unos de otros, por lo que compartir experiencias entre ellos es una buena práctica.

2.3.2 La seguridad es una responsabilidad compartida

En un entorno Agile, se debe integrar la seguridad desde el principio. No hay tiempo en cada *sprint* para crear funciones, hacer pruebas de seguridad y luego corregir las debilidades. Esto requeriría de pruebas completas cada vez, produciendo retrabajo. Debido a esto, la seguridad debe ser responsabilidad de todos, incluido el propietario del producto. Esto pone a los profesionales de la seguridad en un rol de asesoramiento y apoyo. Al incluir la seguridad en las discusiones y en la planificación, no queda espacio para ignorarla.

2.3.3 Campeones de seguridad

Un campeón de seguridad es un enlace entre los desarrolladores y la seguridad de la información. Actúa como punto de contacto para cuestiones relacionadas con la seguridad e intenta eliminar cualquier impedimento que dificulte al equipo el desarrollar software de forma segura. Debido a la colaboración continua en Agile, este rol es muy importante y debe ser asumido por alguien con habilidades de comunicación y aprendizaje.

2.3.4 Estrategia y métricas

En metodología Agile, los ciclos de retroalimentación son cortos y la verificación es frecuente. No se puede controlar lo que no se mide; por lo que, para mejorar la madurez, la calidad de los resultados y la seguridad es beneficioso medir estos aspectos para tomar decisiones.

2.3.5 Modelado de amenazas incrementales

Aunque en metodología Agile se establece que *"el software funcional es más importante que la documentación completa"*, el concepto de modelo de amenaza no se debe descartar [8]. En primer lugar, un modelo de amenaza proporciona un patrón de posibles ataques que disminuye la probabilidad de errores de seguridad. En segundo lugar, un modelo de amenaza puede ser útil para seleccionar los requisitos correctos: genéricos del sistema y específicos de la historia.

No es necesario repetir todo el proceso de modelado de amenazas cada vez. Se puede hacer de forma incremental a medida que evoluciona el software, evaluando para cada nueva historia si en ésta se introduce un nuevo componente, proceso, flujo de datos o límite de confianza.

2.3.6 Selección y preparación de requisitos.

En un proceso Agile, se debe seleccionar y probar algunos requisitos para cada *sprint*. Para que esto sea posible, es necesario seleccionar y preparar los requisitos relevantes por sistema. El objetivo es hacer que el proceso de la historia sea eficiente y proporcionar un conjunto mínimo de requisitos relevantes para cada situación, incluidas las instrucciones para los desarrolladores junto con los planes de prueba correspondientes.

Antes de que se dé comienzo al desarrollo, los requisitos pueden ser seleccionados de un conjunto de requisitos estándar, que normalmente se establecen a nivel de organización, teniendo en cuenta los estándares y el cumplimiento de la industria. Este proceso de selección se basa en el contexto del software (dominio, función, opciones de tecnología de riesgos, etc.) y se debe revisar cuando cambia este contexto. El análisis de riesgos es una parte típica de este proceso. Por cada requerimiento de seguridad, existen las siguientes opciones en la selección y preparación [7]:

- No aplicable en el contexto.
- Necesita instrucciones genéricas del sistema o específicas de la historia, además de las pruebas correspondientes.
- Aplicable, pero con riesgo aceptado.
- Cubierto por la tecnología utilizada.

Los requisitos de seguridad específicos deben estar disponibles en una lista de selección, agrupados por desencadenantes, que permiten la selección posterior de los requisitos correctos según el tipo de trabajo hecho en una historia. Los requisitos específicos de la historia elegidos pasan a formar parte de los criterios de aceptación de una historia en forma de instrucciones para los desarrolladores y las pruebas que se deben realizar.

En resumen: los entregables de la selección y preparación de requisitos de seguridad son:

- Una lista de requisitos que se deben aplicar por historia, en función de los factores desencadenantes.
- Una lista de requisitos que siempre deben aplicarse, independientemente de la historia.
- Una selección de requisitos que se deben incluir en la capacitación del equipo de desarrollo.
- Una lista de requisitos que se consideran riesgos que no aplican o que son aceptables (que pueden basarse en equilibrar la seguridad con el esfuerzo o compromisos en la usabilidad/rendimiento, etc.)

Es muy importante tener los requisitos de seguridad disponibles para seleccionarlos cuando se realizan tipos específicos de trabajo. Este enfoque de fortalecimiento de la seguridad (higiene de la seguridad, por así decirlo) complementa muy bien el enfoque basado en el riesgo del modelado de amenazas como otra perspectiva para ver cómo se debe incorporar la seguridad.

2.3.7 Requisitos en las historias

Los requisitos de seguridad relevantes deben ser incluidos en las historias para que, de este modo, sean parte de los criterios de aceptación. Esto proporciona la información necesaria para la planificación del trabajo. El

refinamiento de las historias se realiza durante su creación y durante el refinamiento del *backlog*, si es necesario con la ayuda de un experto en seguridad. Existen diferentes maneras de recopilar los requisitos:

- **Uso de activadores:** en caso de que los requisitos en una lista de selección se preparen con “*activadores*”, estos se pueden usar para identificar si los requisitos se ajustan a la historia (por ejemplo, al almacenar contraseñas: aplicar *hash* específico).
- **Uso de la experiencia:** la experiencia en seguridad puede ayudar a seleccionar de manera eficiente los requisitos adecuados con una historia.
- **Usar historias de abuso (cómo se puede atacar el sistema):** pueden ayudar a identificar las debilidades y, por lo tanto, vincularlas con los requisitos correctos.
- **Uso de ejercicios de modelado de amenazas.**

Los “*activadores*” son descripciones de funcionalidad o arquitectura que necesitan conjuntos específicos de requisitos. Por ejemplo: “*formularios de usuario*” (para requisitos sobre la validación de la entrada del usuario), “*sesión*” (para requisitos sobre *tokens*, tiempo de espera, etc.) o “*almacenamiento de contraseñas*” (para requisitos sobre *hashing*).

La clave de la seguridad Agile es aplicar los requisitos adecuados en el momento adecuado. Si las historias no mencionan los requisitos de seguridad, estos son olvidados con facilidad, quedando fuera de la planificación del trabajo.

La selección de requisitos debe ser realizada durante la creación de la historia o durante las mejoras del trabajo pendiente. No se recomienda hacer esto durante la planificación del *sprint*.

2.3.8 Historias de abuso

Una historia de abuso es una descripción, desde la perspectiva de un atacante, de cómo se puede abusar de un sistema a través de una debilidad de seguridad. En general, no se considera dentro del *sprint* porque no describe un trabajo de desarrollo específico que se pueda planificar. El propósito de una historia de abuso es ayudar al equipo a pensar en aquello que podría salir mal para especificar el trabajo y los requisitos necesarios en las historias de usuario.

Las historias de abuso son ventajosas principalmente al comienzo del desarrollo. Estas pueden ser útiles como antecedentes para algunas pruebas de seguridad, explicando, por ejemplo, a un *pentester* contra qué amenazas se debe probar un requisito específico.

2.3.9 Pruebas ágiles

En Agile, las pruebas se realizan con más frecuencia y, por lo general, cubren las mismas partes de un sistema una y otra vez, por lo que es importante automatizar las pruebas tanto como sea posible y limitar las pruebas manuales en función de los cambios realizados.

La automatización de las pruebas de seguridad se puede realizar mediante la programación de pruebas a nivel de unidad e integración, y mediante la implementación de herramientas de prueba. Las categorías importantes de herramientas de prueba son herramientas de verificación de dependencia (SCA), herramientas de prueba dinámica de seguridad de aplicaciones (DAST), herramientas de análisis estático (SAST). La salida de SAST y las herramientas de dependencia suelen contener muchos falsos positivos, por lo que se debe realizar una verificación de los resultados.

No todas las pruebas se pueden automatizar. Por ejemplo, las herramientas SAST están limitadas a ciertas categorías de problemas en el código. Las pruebas manuales necesarias que consumen mucho tiempo se pueden minimizar basándose en los efectos esperados de los cambios. Para la revisión del código, esto significa que la revisión debe centrarse solo en el código modificado y agregado y, por lo tanto, requiere de un enfoque y una administración estructurados. Para el *pentesting* manual, esto significa que la prueba debe centrarse en el comportamiento que probablemente se verá afectado por los cambios realizados. Debido a que el efecto de los cambios en el comportamiento es difícil de predecir, se pueden cometer errores al limitar el *pentesting*, por lo que se recomienda realizar una prueba de penetración completa cuando el riesgo de tal error se considere alto.

2.4 Trabajos relacionados

Como ya se ha mencionado, existe una preocupación constante dentro de las organizaciones por mejorar la seguridad de su software y, en el caso de las empresas del sector financiero, por mejorar el nivel de cumplimiento de las normas a las que se encuentran sujetas. Es así como se han desarrollado diferentes investigaciones relacionadas con estos temas y que aportan a una mejor comprensión para el desarrollo de este trabajo.

Entre los trabajos encontrados en la revisión bibliográfica realizada se tiene:

El trabajo de investigación de González Peña (2018) [9] corresponde a un estudio práctico de análisis de vulnerabilidades en algunas de las aplicaciones web con que cuenta el Instituto Nacional de Estadística y Geografía (INEGI) de la República de México. Esta institución genera información sobre fenómenos demográficos, sociales, económicos y del medio ambiente.

En este trabajo práctico se elabora una evaluación en materia de seguridad en el organismo gubernamental con proyectos e información real sobre las amenazas o vulnerabilidades que se pueden presentar, en un ambiente controlado. Para esto, como primera etapa se usa el modelo de madurez SAMM de OWASP para evaluar a la organización y establecer el estado inicial en temas de seguridad. El estudio se centra principalmente en la función de negocio de “Verificación” y las actividades de, “Revisión del Diseño, Revisión de la Implementación y Pruebas de Seguridad”.

Este trabajo, a pesar de que aborda solo una función de negocio del modelo SAMM de OWASP versión 1, resulta bastante interesante ya que presenta una aplicación práctica del modelo para la elaboración de un plan de actividades para la evaluación de la seguridad de un conjunto de aplicaciones. Además, se presentan resultados obtenidos mediante el uso de la herramienta de código abierto OWASP Zed Attack Proxy, la cual es una de las herramientas propuestas dentro de la metodología para el mejoramiento del cumplimiento de las normas PCI-DSS.

Por otro lado, se debe señalar que durante este estudio no se ejecutaron actividades para crear un ciclo de desarrollo seguro para mejorar la situación; sin embargo, el autor sí recomienda a la organización analizada la utilización completa del modelo OWASP SAMM en el futuro.

El objetivo del trabajo de S. D. Calderón (2020) [10] fue diseñar una guía para establecer el estándar PCI DSS V.3.2.1 en una empresa concentradora de pagos, determinando las consideraciones necesarias para su certificación y sostenimiento; en cumplimiento a las exigencias de la Superintendencia Financiera de Colombia y las necesidades de los clientes/usuarios en sus procesos transaccionales y de comercio electrónico.

El aspecto de mayor relevancia de este trabajo consiste en la aplicación de la metodología de análisis de riesgos Octave Allegro para el análisis de riesgos, amenazas y vulnerabilidades que están asociados al cumplimiento del estándar PCI-DSS V3.2.1. Además, sugiere mitigaciones a los riesgos detectados y recomendaciones para lograr la certificación correspondiente.

Esta guía fue estructurada esencialmente para un concentrador de pagos, pero puede ser adaptada a cualquier organización que tenga características como proveedor de servicio en el manejo de datos de cuentas (CHD y/o SAD)¹⁰; a fin de que puedan acoplar los requisitos del estándar, las recomendaciones de seguridad y demás aspectos como las fases de evaluación para poder lograr la certificación en el estándar PCI-DSS si éste es requerido en alguna organización.

En cuanto al desarrollo de software, esta guía solo presenta recomendaciones para la evaluación de códigos y análisis de vulnerabilidades, sin profundizar en este punto.

¹⁰ CHD = Acrónimo de "Cardholder Data"; consta del número de cuenta principal, el nombre del titular de la tarjeta, la fecha de vencimiento de la tarjeta y, en ocasiones, el código de servicio.

SAD = Acrónimo de "Sensitive Authentication Data", incluye la información de la pista magnética, el número confidencial o clave (PIN), así como el valor de autorización de tarjeta no presente al que se denomina como CVV2.

El modelo de ciclo de desarrollo seguro basado en SAMM establece una etapa de análisis y mitigación de riesgos para la construcción de productos de software. Esta guía sirve de referencia para que las organizaciones realicen esta actividad.

En el trabajo de Z. A. Calle, A. J. Mejía (2015) [11] se ejecuta un análisis de la implementación del estándar PCI-DSS como una propuesta para reforzar la seguridad de información y garantizar el correcto procesamiento de tarjetas de crédito y débito dentro de un banco.

El objetivo de este análisis es demostrar la importancia de la implementación del estándar, definiendo la estructura del mismo, y detallando la garantía que proporciona sobre los datos del titular de la tarjeta.

En todos y cada uno de los trabajos descritos se manifiesta la necesidad de mejorar la seguridad de datos y aplicaciones. Para ello se presentan diferentes enfoques y la utilización de normas y estándares existentes para este propósito. Algunos de ellos utilizan el modelo SAMM para realizar una evaluación de la situación actual de las organizaciones bajo estudio y abordar algunos de los requisitos PCI-DSS no relacionados directamente con el desarrollo de software. Otros proponen guías o revisiones que permitan mejorar los niveles de cumplimiento. Sin embargo, ninguno de ellos hace uso del modelo SAMM para intervenir los procesos de desarrollo de software de las organizaciones e intentar conseguir mejorar los niveles de seguridad.

Cada trabajo es, sin lugar a duda, un aporte para el mejoramiento de la seguridad de datos y aplicaciones al interior de las organizaciones y representa una fuente relevante de información para cualquiera que se encuentre interesado en el tema.

3. Propuesta de Solución y Caso de Estudio

3.1 Contexto

El proyecto de estudio se origina debido a la necesidad de mejorar el proceso de desarrollo de software en el área de desarrollo de la empresa Multicaja, interviniendo éste e incorporando actividades y elementos relacionados con el modelo SAMM. Esta intervención tiene por finalidad mejorar el cumplimiento de los estándares y normas de seguridad PCI-DSS, las que son una exigencia en la industria a la que pertenece la empresa.

3.2 Situación Actual

A continuación se describen las áreas que participan en el ciclo de desarrollo de productos de software y las actividades realizadas por cada una de ellas. Las áreas son las siguientes:

- Procesos y Negocios
- Desarrollo
- QA y Testing
- Operaciones TI

3.2.1 Procesos y Negocios

El área de procesos y negocios se encarga de documentar los requerimientos e inquietudes de quienes han ideado el producto y de los futuros usuarios. Su tarea principal es reunir antecedentes y detallar las especificaciones para que se pueda dar inicio a la construcción del producto de software.

Las actividades desarrolladas por esta área son las siguientes:

- Definir requerimientos
- Definir criterios de aceptación
- Entregar documentación e historias de usuario

3.2.2 Área de Desarrollo

El área de desarrollo es la encargada de la construcción de los productos de software, y se encuentra organizada por células ágiles compuestas por un *scrum master* y un grupo de cinco a seis desarrolladores. Además, cada célula tiene asignado un arquitecto de dedicación parcial y uno o dos analistas de calidad de dedicación completa.

Las actividades que debe realizar esta área son las siguientes:

- Revisar historias de usuario
- Definir tiempos y recursos
- Construcción
- Verificar calidad del código
- Subir códigos a repositorio
- Desplegar en ambiente de desarrollo
- Probar desarrollo
- Documentar paso
- Solicitar pruebas a QA

3.2.3 Área de QA y Testing

El área de QA y Testing se encarga de realizar pruebas de calidad al software entregado por las células de desarrollo.

Las actividades de las que se encarga esta área son las siguientes:

- Planificar pruebas QA
- Desplegar en ambiente QA
- Ejecutar pruebas
- Revisar documentación del paso
- Entregar a producción
- Solicitar validación del usuario
- Solicitar vuelta atrás

En la actualidad, el modelo de trabajo considera analistas de calidad que se encuentran asignados con dedicación completa a las células. De este modo, se pretende cumplir de mejor manera con los principios de la metodología ágil, cuya implementación es uno de los objetivos de la empresa.

3.2.4 Área de Operaciones TI

El área de operaciones TI es la encargada de la continuidad operacional de los sistemas y procesos de software que se encuentran en ambientes de producción.

Dentro de sus labores se encuentra el monitoreo de sistemas y procesos, declaración de incidencias y la ejecución de pasos a ambientes productivos.

Dentro de las actividades de las que se encarga esta área, se encuentran:

- Revisar documentación del paso
- Planificar ejecución del paso
- Ejecutar paso
- Ejecutar vuelta atrás
- Informar vuelta atrás

Una descripción detallada de las actividades que desarrolla cada área se encuentra en la sección “*Anexo 2 Descripción de actividades que forman parte del ciclo de desarrollo de productos de software al interior de la organización*”.

3.3 Evaluación de la situación actual

Evaluar la situación actual de la organización permite priorizar qué actividades propuestas por el modelo SAMM pueden ser abordadas de acuerdo con los objetivos, necesidades y recursos disponibles.

Para establecer el punto de partida o situación actual de una organización en cuanto a su postura de seguridad, el modelo SAAM de OWASP ofrece herramientas para llevar a cabo una autoevaluación. La herramienta más simple consiste en una planilla de cálculo (SAMM Assessment spreadsheet.xlsx), preparada por la organización OWASP¹¹ en la que se presenta un listado de 90 preguntas para evaluar el estado de madurez de las diferentes actividades del modelo. Para cada pregunta se presenta un conjunto de criterios de calidad. Además, la hoja de cálculo presenta las posibles respuestas a cada una de las preguntas.

Para establecer el nivel actual de madurez de la organización bajo estudio, se entregó a 13 miembros de la organización (6 desarrolladores, 2 líderes técnicos, 2 ingenieros de procesos, 1 arquitectos y 2 analistas de QA)

¹¹ La planilla con las 90 preguntas preparadas por la organización OWASP para evaluar el nivel de madurez SAMM se encuentra disponible en <https://owasp-samm.org/assessment/>

el cuestionario de preguntas definido en la planilla de autoevaluación.

Una descripción de la forma de establecer los niveles de madurez para cada práctica de seguridad se encuentra en la sección “Anexo 3 Cálculo de niveles de madurez de SAMM”.

3.3.1 Resultados de autoevaluación y niveles de madurez

Analizando las respuestas entregadas por los miembros de los equipos de trabajo a las preguntas indicadas en la planilla de autoevaluación del modelo y, recopilando información respecto de las actividades internas de seguridad que actualmente se realizan al interior de la organización bajo estudio, se tiene que los actuales niveles de madurez son los siguientes:

Tabla 4. Resultados de autoevaluación

Puntaje de madurez actual						
				Índice de Madurez		
Funciones de negocio	Nivel Actual	Prácticas de seguridad	Nivel Actual	1	2	3
Gobierno	0,88	Estrategia y Métricas	1,01	0,38	0,38	0,25
		Política y cumplimiento	0,75	0,25	0,25	0,25
		Formación y orientación	0,88	0,5	0,25	0,13
Diseño	1,05	Evaluación de amenazas	0,75	0,25	0,25	0,25
		Requisitos de seguridad	1,39	0,38	0,63	0,38
		Arquitectura de seguridad	1,01	0,25	0,38	0,38
Implementación	1,17	Construcción segura	1,38	0,38	0,5	0,5
		Despliegue Seguro	1,38	0,5	0,5	0,38
		Gestión de defectos	0,75	0,25	0,25	0,25
Verificación	0,38	Evaluación de la Arquitectura	0,75	0,25	0,25	0,25
		Pruebas Orientadas a Requisitos	0,13	0,13	0	0
		Pruebas de seguridad	0,26	0,13	0	0,13
Operación	1,30	Gestión de Incidentes	1,76	0,63	0,63	0,5
		Gestión del Entorno	1,14	0,38	0,38	0,38
		Gestión operativa	1,01	0,63	0,25	0,13

Al hacer una revisión de los resultados (Figura 2), se puede constatar que los niveles actuales de madurez de la organización en torno a actividades de seguridad de software son bajos, a pesar de que existen iniciativas en este sentido. Lo anterior se explica debido a que muchas de estas iniciativas no han sido completadas o comunicadas a todo el personal y/o sólo se circunscriben a un conjunto de personas.

Por otro lado, se tiene que los equipos de desarrollo, en algunos casos, se encuentran dedicados a la entrega de productos que cumplan con los requisitos funcionales solicitados por los usuarios, dejando que la seguridad se encuentre determinada por la aplicación de algunas herramientas automatizadas.

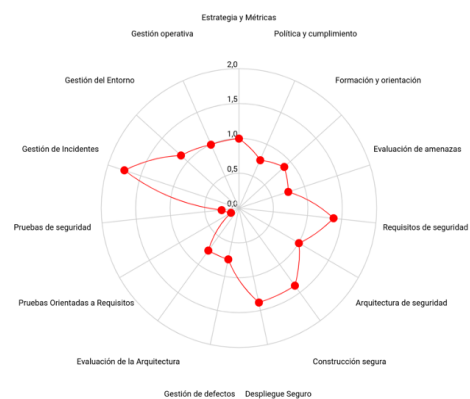


Figura 2. Actuales niveles de madurez de acuerdo a OWASP SAMM.

3.4 Metodología propuesta para el desarrollo de software seguro

Para mejorar la seguridad de los productos de software desarrollados al interior de la organización, se propone intervenir el proceso actual de desarrollo, incorporando elemento establecidos en el modelo OWASP SAMM, así como las indicaciones descritas en la sección “2.3 Metodología Agile y Modelo de Madurez SAMM” con el fin de mejorar los niveles de madurez actuales. Además, también se pretende que los equipos de trabajo conozcan y mejoren el nivel de cumplimiento de las normas PCI-DSS.

Esta intervención consiste en agregar al proceso actual 11 actividades, que forman parte del modelo SAMM,

las que fueron seleccionadas considerando la relevancia que éstas tendrían para mejorar la situación actual del proceso de desarrollo, su aporte en la seguridad de los productos, su aporte al cumplimiento normativo (PCI-DSS) y la posibilidad de ser realizadas sin un aumento significativo en el uso de recursos o incremento en los tiempos de trabajo. Por otra parte, también se considera la existencia, para algunas de ellas, de herramientas automatizadas de código abierto que permiten hacer evaluaciones rápidamente.

Las actividades agregadas al proceso actual son las siguientes:

- | | |
|---|--|
| 1. Evaluar el riesgo de la aplicación | 7. Actualizar inventario de componentes |
| 2. Realizar modelado básico de amenazas | 8. Probar requisitos no funcionales |
| 3. Identificar requisitos de seguridad | 9. Ejecutar pruebas de seguridad en ambiente de desarrollo |
| 4. Integrar en el proceso los requisitos de seguridad | 10. Probar requisitos no funcionales en ambiente de QA |
| 5. Evaluar la arquitectura de la aplicación | 11. Ejecutar pruebas de seguridad en ambiente de QA |
| 6. Identificar y evaluar herramientas y tecnologías | |

Integrando estas 11 actividades, en el actual proceso de desarrollo, el listado completo de actividades que componen la metodología propuesta es el siguiente:

Tabla 5. Actividades, Procesos y Negocio, Desarrollo y QA (Se destacan actividades agregadas).

Actividad	Descripción
3.4.1 Definir requerimientos funcionales	Se deben especificar, de manera clara y concisa, los servicios que prestará el software y la forma en que éste tiene que reaccionar ante determinadas entradas.
3.4.2 Evaluar el riesgo de la aplicación.	Se debe utilizar un método sencillo para valorar los riesgos de la aplicación/servicio. Los riesgos deben ser evaluados, estimando el impacto potencial que supone para el negocio de la organización.
3.4.3 Realizar modelado básico de amenazas para la aplicación.	Se debe identificar, evaluar y gestionar las amenazas del sistema, los defectos de diseño en la arquitectura del mismo y las mitigaciones de seguridad recomendadas.
3.4.4 Identificar requisitos de seguridad.	Se deben identificar de manera específica los requisitos de seguridad del software a partir de los requisitos funcionales del proyecto. En la sección "Anexo 1 Requisitos de seguridad extraídos desde las normas PCI DSS" del Anexo se presenta un listado de requisitos de seguridad exigidos por la normativa PCI DSS.
3.4.5 Integrar en el proceso los requisitos de seguridad.	Los requisitos de seguridad deben ser gestionados de la misma manera que los requisitos funcionales. Para ello es necesario incluirlos en los documentos de análisis de requisitos, creación de historias de usuario (si se trabaja con metodología Agile), etc. Es decir, proceder de la misma forma que con los requisitos funcionales.
3.4.6 Definir criterios de aceptación funcionales y de seguridad	Dentro de los criterios de aceptación se debe incorporar escenarios relacionados no sólo con los requisitos funcionales, sino también con el comportamiento de la aplicación frente a requisitos de seguridad.
3.4.7 Entregar documentación e historias de usuario (incluir requisitos de seguridad)	Como ya se ha mencionado, los requisitos de seguridad deben ser gestionados de la misma forma que se hace con los requisitos funcionales. Por lo tanto, éstos deben ser documentados e incluidos dentro de las historias de usuario que son entregadas a los equipos de desarrollo para su evaluación, priorización, estimación de esfuerzos, asignación, etc.

Las actividades 3.4.1 hasta 3.4.7 requieren de la participación de usuarios, ingenieros de negocio, analistas de calidad y desarrolladores (Tabla 5).

Tabla 6. Actividades área de Desarrollo (Se destacan actividades agregadas).

Actividad	Descripción
3.4.8 Revisar historias de usuario	Se debe ejecutar una revisión de las historias de usuario y de los criterios de aceptación para validar su consistencia con los requisitos funcionales y de seguridad. Además, se debe verificar que dentro de los antecedentes se consideran requisitos de seguridad consistentes con las normas y estándares exigidos a la organización.
3.4.9 Evaluar la arquitectura de la aplicación.	Se debe evaluar la arquitectura de alto nivel de la aplicación y establecer si ésta cumple con las medidas de seguridad básicas.

Actividad	Descripción
3.4.10 Identificar y evaluar herramientas y tecnologías.	El objetivo de esta actividad es tener una visión clara de las herramientas y tecnologías empleadas en el desarrollo y crear conciencia de cómo estas pueden introducir riesgos de seguridad en el producto final.
3.4.11 Definir tiempos y recursos.	Una vez hechas las evaluaciones correspondientes y aceptadas las historias de usuario se procede a acordar los plazos y los recursos necesarios para su construcción.
3.4.12 Construcción	Durante la etapa de construcción el desarrollador procede a la creación de los componentes de software necesarios para cumplir con las historias de usuario.
3.4.13 Actualizar inventario de componentes.	A medida que el proyecto avanza, el equipo de desarrollo debe actualizar el inventario de componentes que forman la aplicación/servicio.
3.4.14 Verificar calidad del código	El objetivo de la etapa es evaluar el código para detectar errores, fallas y normas de calidad establecidas para la organización.
3.4.15 Subir códigos a repositorio	Los códigos que pasan los criterios de calidad deben ser almacenados en un repositorio central que permita su versionamiento.
3.4.16 Desplegar en ambiente de desarrollo	Para verificar que el aplicativo se desplegará correctamente en los ambientes de calidad y producción, los desarrolladores deben realizar una prueba de despliegue en ambiente de desarrollo por medio de la herramienta de despliegue continuo.
3.4.17 Probar requisitos funcionales	Los desarrolladores tienen que ejecutar pruebas funcionales del software. Las pruebas deben estar acorde con los requisitos definidos en la etapa de definición de criterios de aceptación.
3.4.18 Probar requisitos no funcionales.	Los desarrolladores tienen que ejecutar pruebas relacionadas con requisitos no funcionales del software. Las pruebas deben estar acorde con los requisitos definidos en la etapa de definición de criterios de aceptación.
3.4.19 Ejecutar pruebas de seguridad.	Las pruebas de seguridad de las aplicaciones se pueden llevar a cabo mediante la inspección del código fuente de una aplicación sin ejecutarla (análisis estático), u observando el comportamiento de la aplicación en respuesta a diversas condiciones de entrada (análisis dinámico).
3.4.20 Documentar paso	Una vez que el desarrollador finaliza sus pruebas satisfactoriamente, se elabora la documentación necesaria para el despliegue del nuevo aplicativo en los ambientes de pruebas de calidad (QA y Testing) y producción.
3.4.21 Solicitar pruebas a QA	La documentación del paso se sube a la plataforma de gestión de proyectos, se informa la entrega al área de QA y se asigna a él o los analistas correspondientes para la planificación de las pruebas.

Las actividades **3.4.8** a **3.4.21** deben ser realizadas por la cédula de desarrollo, ya que éstas se relacionan directamente con la construcción del software (Tabla 6).

Tabla 7. Actividades área de QA (Se destacan actividades agregadas).

Actividad	Descripción
3.4.22 Planificar pruebas QA	El o los analistas de calidad de software asignados a la célula reciben las solicitudes y deben planificar los recursos necesarios para la ejecución de las pruebas en ambiente de calidad.
3.4.23 Desplegar en ambiente QA	Antes de proceder con la ejecución de las pruebas de calidad, el software desarrollado tiene que ser instalado en el ambiente correspondiente por medio de la herramienta de despliegue continuo.
3.4.24 Probar requisitos funcionales	Los analistas tienen que ejecutar pruebas funcionales del software. Las pruebas deben estar acorde con los requisitos definidos en la etapa de definición de criterios de aceptación.
3.4.25 Probar requisitos no funcionales.	Los analistas tienen que ejecutar pruebas relacionadas con requisitos no funcionales del software. Las pruebas deben estar acorde con los requisitos definidos en la etapa de definición de criterios de aceptación.
3.4.26 Ejecutar pruebas de seguridad.	Al igual que en la actividad 3.4.19 los analistas de calidad deben ejecutar pruebas de seguridad, esta vez con mayor rigurosidad que las hechas por los desarrolladores para, de esta manera, aplicar una segunda revisión que garantice que el software cumple con los requisitos de seguridad establecidos.
3.4.27 Revisar y corregir documentación del paso	Una vez que se han recopilado las evidencias sobre el correcto funcionamiento del software y se tiene la conformidad del usuario con las pruebas, los analistas de calidad

Actividad	Descripción
	tienen que hacer una nueva revisión de la documentación de paso a producción.
3.4.28 Entregar a producción	Una vez que la documentación se encuentra verificada, se procede a enviar la solicitud de paso a producción al área de Operaciones TI, que es la encargada del despliegue de los aplicativos en los ambientes productivos.
3.4.29 Solicitar validación del usuario	Una vez que el paso a ambiente productivo se ha ejecutado, el analista de calidad solicita al usuario correspondiente que haga una validación de las funcionalidades del software entregado.
3.4.30 Solicitar vuelta atrás	En caso de que el usuario indique errores al hacer sus validaciones en ambiente de producción se solicita la ejecución de vuelta atrás del paso a producción al área de Operaciones TI.

Las actividades **3.4.22** a **3.4.30** deben ser realizadas por los analistas de calidad (Área de QA). Estas se relacionan con las pruebas de calidad del software y la verificación de los requisitos funcionales, no funcionales y de seguridad (Tabla 7).

Tabla 8. Actividades área de Operaciones TI

Actividad	Descripción
3.4.31 Revisar documentación del paso	El despliegue del software en ambiente de producción debe ser ejecutado por un ingeniero de operaciones TI, quien revisa la documentación correspondiente en la plataforma de gestión de proyectos para verificar que los pasos a seguir se encuentren debidamente explicados.
3.4.32 Planificar ejecución del paso	El ingeniero de operaciones TI debe planificar el momento adecuado para la ejecución del despliegue, ya que éste puede afectar algunos servicios e incidir en las operaciones en línea y/o en el procesamiento de datos.
3.4.33 Ejecutar paso	El ingeniero de operaciones TI tiene que proceder al despliegue automático del software por medio de la herramienta de despliegue continuo, validar la ejecución mediante la revisión de logs e informar a los interesados.
3.4.34 Ejecutar vuelta atrás	La ejecución de la vuelta atrás se produce en situaciones en las cuales se presentan errores durante el despliegue de los aplicativos en ambientes productivos o cuando el usuario informa errores en el funcionamiento de éstos.
3.4.35 Informar vuelta atrás	La vuelta atrás debe ser informada a los interesados para que estén enterados de la situación y corrijan la causa de los errores presentados. Además, se debe verificar que el estado de los sistemas y/o procesos, que se encuentran en producción, no se haya visto afectado por la falla del paso.

Las actividades **3.4.29** a **3.4.34** deben ser realizadas por personal del área de Operaciones TI. Estas se relacionan con la instalación del software en el ambiente productivo (Tabla 8).

Una descripción detallada de las actividades que componen la metodología propuesta, su relación con el modelo SAMM de OWASP y con los requisitos de seguridad PCI-DSS se presenta en el Anexo “*Anexo 4 Descripción de actividades de la metodología propuesta*”.

Como se ha mencionado en la sección “*2.3 Metodología Agile y Modelo de Madurez SAMM*”, la aplicación del modelo sirve para reforzar la metodología Agile. Para ello es necesario hacer uso de herramientas que faciliten y aceleren las actividades del proceso.

En la sección del Anexo “*Anexo 5 Soluciones y herramientas aplicables*” se presentan referencias a herramientas que pueden ser utilizadas para apoyar la ejecución de algunas de las actividades descritas en la metodología de desarrollo propuesta.

4. Validación de la metodología propuesta

4.1 Evaluación de la metodología propuesta en relación al modelo SAMM de OWASP

Una vez presentada la propuesta metodológica a un equipo de trabajo, y ejecutada una prueba para un proyecto sencillo, se procede nuevamente a dar respuesta al cuestionario de preguntas para evaluar el estado de madurez

de acuerdo al modelo SAAM de OWASP.

4.1.2 Descripción del proyecto de prueba

El proyecto de mantención de tarifas tiene por finalidad permitir la actualización de las comisiones aplicadas a las transacciones de pago con tarjeta. Dentro de los requerimientos funcionales se encuentran:

- Actualización de tarifas por formulario
- La creación de campañas por periodos de tiempo
- La actualización de tarifas por lotes mediante la subida de archivos planos
- El traspaso de los datos a ambiente de procesamiento de transacciones de pago

Se debe garantizar la confidencialidad, integridad y disponibilidad de los datos, por lo que lo hace apropiado para ser desarrollado mediante la aplicación de la metodología propuesta.

4.1.3 Resultados de la nueva autoevaluación

Analizando las respuestas entregadas por los miembros del equipo de trabajo a las preguntas indicadas en la planilla de autoevaluación del modelo SAMM de OWASP, luego del ejercicio de aplicación de la metodología propuesta, se tienen los siguientes resultados:

Tabla 5. Resultados de autoevaluación

Puntaje de madurez actual						
				Índice de Madurez		
Funciones de negocio	Nivel Actual	Prácticas de seguridad	Nivel Actual	1	2	3
Gobierno	1,26	Estrategia y Métricas	1,01	0,38	0,38	0,25
		Política y cumplimiento	1,63	0,63	0,75	0,25
		Formación y orientación	1,13	0,5	0,38	0,25
Diseño	1,88	Evaluación de amenazas	1,63	0,5	0,38	0,75
		Requisitos de seguridad	1,75	0,5	0,75	0,5
		Arquitectura de seguridad	2,25	0,75	0,75	0,75
Implementación	2,33	Construcción segura	2,75	1	1	0,75
		Despliegue Seguro	2,75	1	1	0,75
		Gestión de defectos	1,5	0,5	0,5	0,5
Verificación	1,13	Evaluación de la Arquitectura	1,5	0,5	0,5	0,5
		Pruebas Orientadas a Requisitos	0,88	0,25	0,25	0,38
		Pruebas de seguridad	1,01	0,38	0,25	0,38
Operación	1,30	Gestión de Incidentes	1,76	0,63	0,63	0,5
		Gestión del Entorno	1,14	0,38	0,38	0,38
		Gestión operativa	1,01	0,63	0,25	0,13

Al comparar estos resultados con la autoevaluación anterior sobre la metodología actual que se aplica para el desarrollo de software se tiene que: a juicio del equipo de trabajo, existen prácticas de seguridad que aumentan su nivel de madurez una vez que se aplica la metodología propuesta.

Esto se explica por el hecho de que actualmente no existen, dentro del proceso de desarrollo, actividades relacionadas con la seguridad de los productos definidas formalmente.

La figura 3 presenta una comparación entre niveles de madurez de ambas evaluaciones.

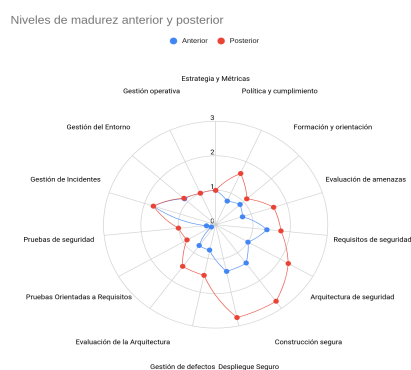


Figura 3. Comparación niveles de madurez (Metodología actual / Propuesta)

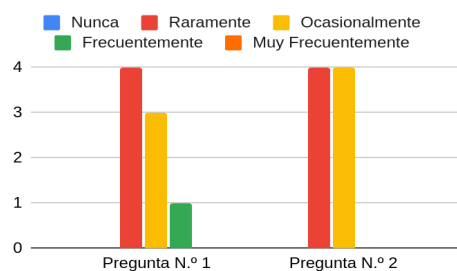
4.2 Evaluación de la metodología propuesta

Para recoger la opinión sobre el trabajo realizado y el aporte que éste representa para los integrantes del equipo de trabajo, se les solicitó dar respuesta a una encuesta, de elaboración propia, de 20 preguntas para medir cual

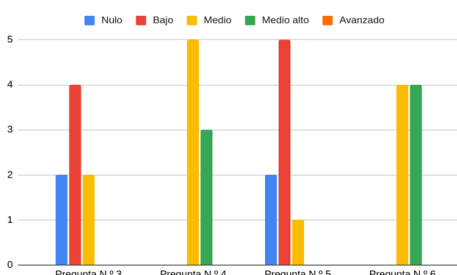
es el grado de relevancia que tienen las nuevas actividades incorporadas dentro del proceso de desarrollo para cada uno de los integrantes.

Este cuestionario fué respondido por un grupo de 9 personas: un líder técnico, seis desarrolladores y dos analistas de calidad de software. El detalle de las opciones de respuesta a cada pregunta, junto con los resultados de la encuesta, se encuentra en el Anexo “Anexo 6 Cuestionario de preguntas sobre la metodología propuesta y sus respuestas”.

Al examinar las respuestas a las preguntas 1 y 2 se puede apreciar que, a juicio de los encuestados, no existe una formación frecuente, al interior de la organización, sobre marcos normativos de la seguridad del software (PCI-DSS), ni sobre ciclos o metodologías de desarrollo seguro de software.

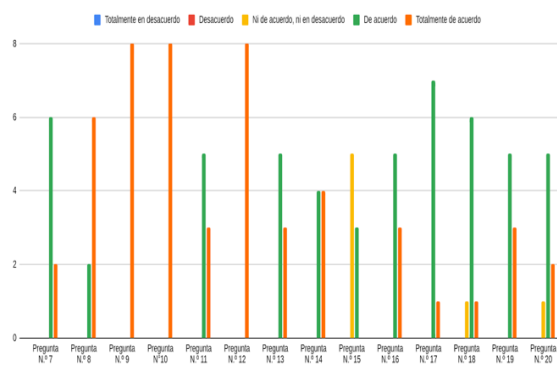


Las respuestas a las preguntas 3 a la 6 representan los resultados obtenidos con relación al aporte en conocimiento normativo y sobre modelos de madurez que, a juicio de los encuestados, aporta el desarrollo de este trabajo de investigación.



Los resultados indican que el trabajo desarrollado ha aportado a mejorar el conocimiento de los miembros del equipo con respecto a las materias que se abordan en éste.

Las respuestas a las preguntas 7 a la 20 representan la opinión del equipo de trabajo respecto a la metodología de desarrollo seguro presentada en esta investigación y de las actividades incorporadas para mejorar la situación actual.



Se puede apreciar que existe una aceptación de que la metodología propuesta representa un aporte que mejora la concientización sobre la seguridad del software, permite la entrega de software más seguro y que su utilización no implica un aumento considerable en el uso de recursos.

Al revisar los resultados obtenidos se puede apreciar que, en general, existe una buena opinión del equipo de trabajo del proceso de desarrollo propuesto. Todos consideran que las actividades realizadas contribuyen a un mejor entendimiento del marco normativo al que se encuentra expuesta la empresa y, en particular a las normas PCI-DSS. También se puede observar que existe un reconocimiento de que las actividades mejoran la concientización de que la seguridad debe ser incorporada dentro del proceso de desarrollo desde el inicio de éste y que contribuyen a la creación de productos más seguros.

Por otro lado, se debe mencionar que a pesar a que, a juicio del equipo, los tiempos no aumentan considerablemente en relación a la metodología actual, si se planteó un grado de inquietud debido a que, para que esto sea posible, se debe hacer uso de herramientas automatizadas para las cuales se requiere de preparación y capacitación.

En resumen, se puede decir que una vez presentada la nueva metodología y ejecutada una pequeña prueba de ésta, el equipo de trabajo considera que, si es posible utilizar una metodología basada en el modelo de madurez

SAMM de OWASP en un proceso de desarrollo basado en agilidad, mejorando la seguridad del software entregado y obviamente reduciendo las incidencias relacionadas directamente con este tipo de fallas. Además, al existir más etapas de análisis, también se mejora la comprensión de los requisitos funcionales.

5. Conclusiones

5.1 Sobre la hipótesis y el trabajo realizado

Durante la ejecución de este trabajo de investigación se realizó la intervención de un proceso de desarrollo dentro de una organización dedicada al procesamiento de pago de cuentas con tarjetas de crédito y débito. Para ello se tomó como referencia el modelo de desarrollo de software seguro SAMM de OWASP.

Durante este proceso se pudo constatar que la utilización de modelo, además de dar como resultado productos de software más seguros y que cumplen en mejor medida con las regulaciones a las que se encuentra sujeta la organización (PCI-DSS), contribuye a mejorar la concientización de los equipos de trabajo en materia de seguridad de software y a lograr un proceso controlado, ordenado y mejor documentado.

Lo anterior es particularmente relevante cuando se trata de procesos de desarrollo en ambientes ágiles, en los cuales se tiende a dejar de lado actividades mencionadas en el modelo que, erróneamente, son consideradas como irrelevantes, que aportan muy poco a los resultados finales o que, definitivamente, contradicen los principios de la agilidad. Sin embargo, mediante una adecuada organización por parte de los equipos de desarrollo y la utilización de herramientas automatizadas, es posible comprobar que dichas actividades si representan un aporte importante al proceso, sin un aumento considerable en el uso de recursos.

Por otro lado, el modelo, al entregar un método de evaluación sobre la postura de seguridad de las organizaciones en materia de seguridad del software, hace posible que éstas sinceren su situación, tomen conciencia y planifiquen mejoras en la materia, alineando los objetivos de seguridad con los objetivos comerciales, de cumplimiento normativo y de crecimiento organizacional.

5.2 Trabajo futuro

A partir de esta investigación, se derivan trabajos futuros y aspectos que requieren una profundización, tanto a nivel organizacional como a nivel de equipos de trabajo:

- En términos organizacionales, es preciso definir una estrategia que permita expandir la utilización del modelo SAMM al resto de la organización para que, de este modo, se logre alcanzar mayores niveles de madurez y de cumplimiento normativo. Lo anterior es de vital importancia ya que esta organización se encuentra en una etapa de crecimiento en sus operaciones comerciales, lo que aumenta los niveles de exigencia a los que se encuentra sometida.
- La estrategia debe promover la participación de los equipos de trabajo en actividades que contribuyan al mejoramiento de los procesos y el aumento de los niveles de madurez de SAMM. Para ello, es fundamental la capacitación constante del personal en herramientas que faciliten el desarrollo de dichas actividades.
- La estrategia debe considerar que las amenazas a la seguridad del software y a la seguridad de la información se encuentran en constante evolución, por lo cual la organización debe programar revisiones periódicas a: modelos de madurez, marcos de trabajo, normativas, capacitaciones constantes en temas de seguridad y todo aquello que permita enfrentar de mejor manera dichas amenazas.

Agradecimientos. Quiero agradecer a mis profesores y compañeros de magíster por los conocimientos y experiencias profesionales compartidas durante este periodo de aprendizaje. Y a mis compañeros de trabajo por el aporte y colaboración en el desarrollo de este trabajo.

Referencias

- [1] Microsoft Company, “About Microsoft SDL”, 2023 [En línea]. Disponible en: <https://www.microsoft.com/en-us/securityengineering/sdl/about>.
- [2] Oracle Company, “Oracle Software Security Assurance”, 2023 [En línea]. Disponible en: <https://www.oracle.com/corporate/security-practices/assurance/>.
- [3] G. McGraw, S. Mígues, J. West “Building Security in Maturity Model. Release 5.0”, octubre de 2013.
- [4] OWASP Foundation, “OWASP SAMM”, Versión 2.0, January 2020.
- [5] PCI (industria de tarjetas de pago), “Normas de seguridad de datos, Requisitos y procedimientos de evaluación de seguridad”, Versión 3.2.1, mayo de 2018.
- [6] Software Engineering Institute (SEI), CMMI® for Development, Version 1.3, Improving processes for developing better products and services, Technical Report CMU/SEI-2010-TR-033 ESC-TR-2010-033, 2010.
- [7] R. Van Der Veer, “SAMM Agile Guidance”, 2019 [En línea]. Disponible en: <https://owaspamm.org/guidance/agile/>.
- [8] M. Beedle, A. Van Bennekum, “Manifesto for Agile Software Development”, 2001 [En línea]. Disponible en: <https://agilemanifesto.org/iso/en/manifesto.html>.
- [9] A.T. González Peña, "Security Testing Techniques and Tools: A Study for Web Based Applications", Tesis, Departamento De Sistemas De Información, Universidad Autónoma de Aguascalientes, Aguascalientes, México, mayo 2018.
- [10] S. D. Calderón Romero, "Guía Para El Cumplimiento uel Estándar PCI-DSS V3.2.1 un una Pasarela ue Pagos", Proyecto De Seguridad Informática, Universidad Nacional Abierta y a Distancia, Bogotá, Colombia, 2020.
- [11] Z. A. Calle, A. J. Mejía, "Análisis de la Implementación del Estándar PCI-DSS en la Seguridad de la Información dentro de dna Institución Financiera", Tesis, Facultad De Ingeniería, Universidad Politécnica Salesiana, Guayaquil, Ecuador, 2015.

Anexos

Anexo 1 Requisitos de seguridad extraídos desde las normas PCI-DSS

Estos requisitos de seguridad se han extraído desde las normas PCI-DSS versión 3.2.1. La numeración corresponde a la entregada en el documento de especificación de la norma.

2.3 Se debe cifrar todo el acceso administrativo que no sea de consola utilizando un cifrado sólido (PCI versión 3.2.1, Página 34).

3.2 No se deben almacenar datos confidenciales de autenticación después de recibir la autorización (aun cuando estén cifrados). Si se reciben datos confidenciales de autenticación, se tiene que convertir todos los datos en irre recuperables al finalizar el proceso de autorización (PCI versión 3.2.1, Página 37).

3.3 Se debe enmascarar el PAN (número de cuenta principal) cuando aparezcan (los primeros seis o los últimos cuatro dígitos es la cantidad máxima de dígitos que debe aparecer), de modo que solo el personal con una necesidad comercial legítima pueda ver más que los primeros seis o los últimos cuatro dígitos del PAN (PCI versión 3.2.1, Página 40).

3.4 Se tiene que convertir el PAN (número de cuenta principal) en ilegible en cualquier lugar donde se almacene (incluidos los datos que se almacenen en medios digitales portátiles, en medios de copia de seguridad y en registros) utilizando cualquiera de los siguientes métodos (PCI versión 3.2.1, Página 42).

4.1 Se debe utilizar criptografía sólida y protocolos de seguridad para proteger los datos confidenciales del titular de la tarjeta durante la transmisión por redes públicas abiertas (PCI versión 3.2.1, Página 51)..

7.1 Se debe limitar el acceso a los componentes del sistema y a los datos del titular de la tarjeta a aquellos individuos cuyas tareas necesitan de ese acceso (PCI versión 3.2.1, Página 75).

7.2 Se tiene que establecer un sistema de control de acceso para los componentes del sistema que restrinja el acceso según la necesidad del usuario de conocer y que se configure para “negar todo”, salvo que se permita específicamente (PCI versión 3.2.1, Página 77).

8.1 Definir e implementar políticas y procedimientos para garantizar la correcta administración de la identificación de usuarios para usuarios no consumidores y administradores en todos los componentes del sistema (PCI versión 3.2.1, Página 78).

8.2 Además de asignar un ID exclusivo, se debe asegurar la existencia de una correcta administración de autenticación de usuarios para usuarios no consumidores y administradores en todos los componentes del sistema (PCI versión 3.2.1, Página 81).

10.1 Implemente pistas de auditoría para vincular todo acceso a componentes del sistema con usuarios específicos (PCI versión 3.2.1, Página 103).

10.2 Implemente pistas de auditoría automáticas en todos los componentes del sistema (PCI versión 3.2.1, Página 103).

10.3 Se debe registrar, al menos, las siguientes entradas de pistas de auditoría de los componentes del sistema para cada evento (PCI versión 3.2.1, Página 105).



- Identificación de usuarios
- Tipo de evento
- Fecha y hora
- Indicación de éxito o fallo
- Origen del evento
- Identidad o nombre de los datos, componentes del sistema o recursos afectado

Anexo 2 Descripción de actividades que forman parte del ciclo de desarrollo de productos de software al interior de la organización

En esta sección se detallan las actividades realizadas por las áreas involucradas en el desarrollo de productos de software al interior de la organización.

Se entregan:

- Listado de actividades
- Diagramas
- Entradas y salidas
- Matriz de responsabilidades

Área de Procesos y Negocios

Actividades	Descripción
Definir requerimientos	Usuarios e ingenieros de negocios se reúnen para acordar los alcances del proyecto y definir las especificaciones que debe cumplir el producto de software. El ingeniero de negocios traduce las especificaciones en historias de usuario, en las que se describe en detalle cada una de las funcionalidades que formarán parte del producto de software.
Definir criterios de aceptación	Los criterios de aceptación corresponden a la descripción del comportamiento que deben tener las funcionalidades desarrolladas frente a diferentes escenarios. Estos antecedentes sirven como guía para las pruebas que se deben hacer a los desarrollos una vez entregados al área de QA y Testing.
Entregar documentación e historias de usuario	Las historias de usuario, junto con la documentación complementaria, son entregadas por los ingenieros de negocios a las células de desarrollo a través de una plataforma de gestión de proyectos. De esta manera, los miembros de las células pueden revisar todo lo concerniente al proyecto.

En el siguiente diagrama se presenta el flujo de actividades realizadas por el equipo del **Área de Procesos y Negocios**

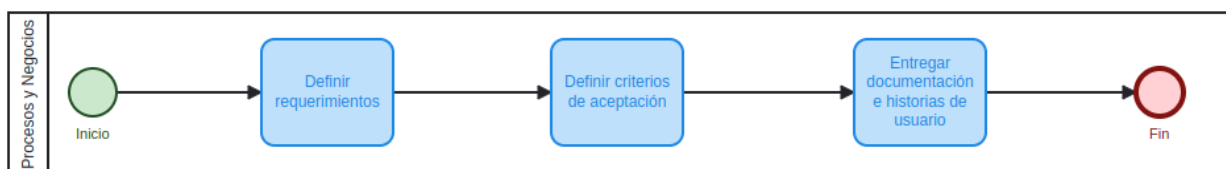


Figura 4. Actividades realizadas por el equipo de Procesos y Negocios

Procesos y Negocios	
Entradas	Salidas
• Ideas o requerimientos entregadas al área por los usuarios	• Se entregan documentos estructurados a Desarrollo con el detalle de los requerimientos, conocidas como historias de usuario.

Área de Desarrollo

Actividades	Descripción
Revisar historias de usuario	Los miembros de la célula de desarrollo deben revisar las historias de usuario para verificar que éstas se puedan interpretar de manera precisa. En caso contrario, la historia es devuelta al ingeniero de negocio que la creó para que sea reformulada.
Definir tiempos y recursos	Una vez aceptada la historia, se procede a acordar los plazos y los recursos necesarios para su construcción. Las historias que requieren mayor esfuerzo son divididas en tareas que son

Actividades	Descripción
	distribuidas entre los miembros de la célula.
Construcción	Durante la etapa de construcción el desarrollador procede a la creación de los componentes de software necesarios para cumplir con las historias de usuario. Para ello hace uso de un IDE (Entorno de Desarrollo Integrado) y de librerías que forman parte del <i>stack</i> tecnológico de la empresa.
Verificar calidad del código	Cada desarrollador debe verificar la calidad del código construido por medio de un escáner que se encuentra integrado al IDE. Esto tiene como finalidad que el código se apegue a las mejores prácticas y que se encuentre libre de errores o vulnerabilidades conocidas.
Subir códigos a repositorio	Los códigos que pasan los criterios de calidad deben ser almacenados en un repositorio central y es responsabilidad del desarrollador mantenerlos actualizados. Este es un requisito que se debe cumplir para que los productos desarrollados puedan ser desplegados en los respectivos ambientes.
Desplegar en ambiente de desarrollo	Para verificar que el aplicativo se desplegará correctamente en los ambientes de calidad y producción, los desarrolladores realizan una prueba de despliegue en ambiente de desarrollo por medio de una herramienta de despliegue continuo. Durante este proceso nuevamente se revisa la calidad del código, ya que la herramienta tiene definida una Quality Gate que no permite desplegar aplicativos cuyos códigos no cumplen con los criterios de buenas prácticas, además de estar libres de vulnerabilidades.
Probar desarrollo	Los desarrolladores ejecutan pruebas funcionales en ambiente de desarrollo para validar que el aplicativo cumple con las especificaciones entregadas en las historias de usuario.
Documentar paso	Una vez que el desarrollador finaliza sus pruebas satisfactoriamente, elabora la documentación necesaria para el despliegue del nuevo aplicativo en los ambientes de pruebas de calidad (QA y Testing) y producción.
Solicitar pruebas a QA	La documentación del paso se sube a la plataforma de gestión de proyectos, se informa la entrega al área de QA y se asigna a él o los analistas correspondientes para la planificación de las pruebas.

En el siguiente diagrama se presenta el flujo de actividades realizadas por el equipo del **Área de Desarrollo**

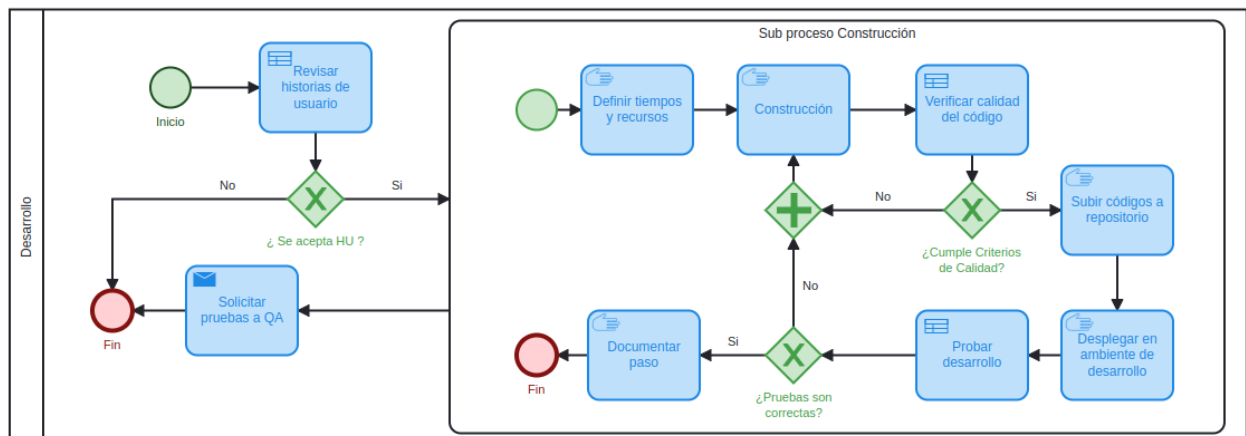


Figura 5. Actividades realizadas por el equipo de Desarrollo

Desarrollo	
Entradas	Salidas
Historias de usuario entregadas por al área de Procesos y Negocios	- Revisan las HU, en caso de identificar inconsistencias se devuelve a Procesos y Negocios - Se entregan los códigos fuente de la aplicación, el documento de paso a ambiente de producción a QA y se solicitan las pruebas en ambiente de QA

Área QA y Testing

Actividades	Descripción
Planificar pruebas QA	El o los analistas de calidad de software, asignados a la célula, reciben las solicitudes y planifican los recursos y tiempos necesarios para la ejecución de las pruebas en ambiente de calidad.
Desplegar en ambiente QA	Antes de proceder con la ejecución de las pruebas de calidad, el software desarrollado es instalado en el ambiente por medio de la herramienta de despliegue continuo. Esta actividad es llevada a cabo por los analistas de calidad, quienes deben informar cualquier error que se presente.
Ejecutar pruebas	Los analistas de calidad ejecutan pruebas sobre los aplicativos desarrollados para verificar que éstos cumplen con las historias de usuario y criterios de aceptación definidos. Si las pruebas son satisfactorias, el analista de calidad envía la evidencia de éstas a los usuarios para que ellos informen su conformidad con el desarrollo. En caso contrario, se da aviso a la célula para que se corrijan las fallas encontradas.
Revisar documentación del paso	Si el usuario envía su conformidad con las pruebas, los analistas de calidad revisan y actualizan la documentación de paso a producción. En caso de que ésta presente alguna falencia, informan a la célula para su corrección.
Entregar a producción	Una vez que la documentación se encuentra verificada, se procede a enviar la solicitud de paso a producción al área de Operaciones TI, que es la encargada del despliegue de los aplicativos en los ambientes productivos.
Solicitar validación del usuario	Una vez que el paso a ambiente productivo se ha ejecutado, el analista de calidad solicita al usuario correspondiente que haga una validación de las funcionalidades del software entregado. Si éste confirma el correcto funcionamiento, se da aviso a operaciones TI para que dé por finalizado el paso de manera correcta. En caso contrario, el analista de calidad informa a operaciones TI que se debe ejecutar la vuelta atrás del paso e informar éste como fallido.
Solicitar vuelta atrás	En caso de que el usuario indique errores al hacer sus validaciones en ambiente de producción se solicita la ejecución de vuelta atrás del paso a producción al área de Operaciones TI.

En el siguiente diagrama se presenta el flujo de actividades realizadas por el equipo del Área QA y Testing

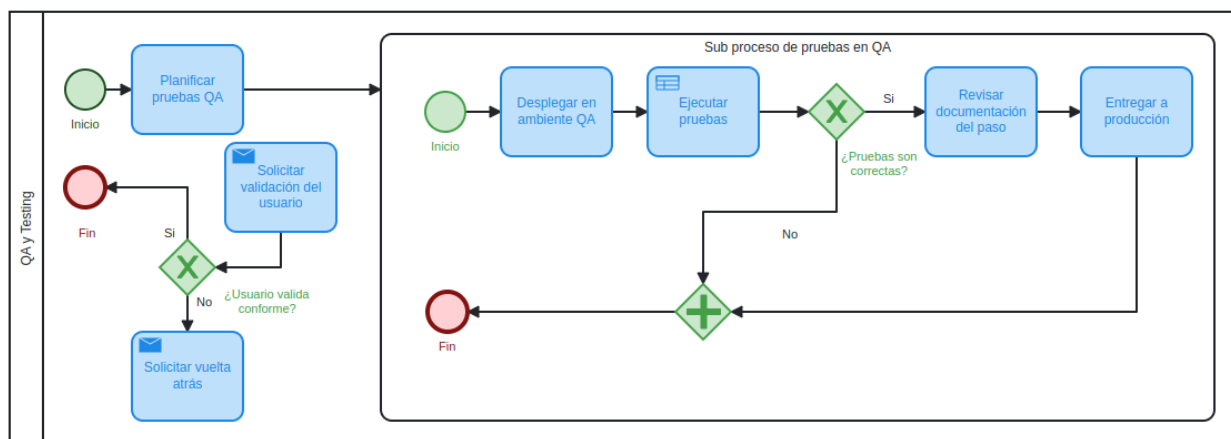


Figura 6. Actividades realizadas por el equipo de QA y Testing

QA y Testing	
Entradas	Salidas

• Historias de usuario entregadas por al área de Procesos y Negocios • Documento de paso a producción • Solicitud de pruebas en ambiente de QA desde Desarrollo	• Se entrega documento revisado a Operaciones TI • Se ejecutan pruebas, en caso de encontrar fallas se informa a Desarrollo para sus correcciones • Se envía a Operaciones TI para la ejecución de la aplicación a ambiente de producción • Se solicita validación usuaria • Se solicita vuelta atrás en caso de el usuario indique fallas en ambiente de producción
---	--

Área de Operaciones TI

Actividades	Descripción
Revisar documentación del paso	El despliegue del software en ambiente de producción es ejecutado por un ingeniero de operaciones TI, quien revisa la documentación correspondiente en la plataforma de gestión de proyectos para verificar que las acciones a seguir se encuentren debidamente explicadas. En caso contrario, suspende la ejecución del despliegue hasta que la documentación sea corregida.
Planificar ejecución del paso	El ingeniero de operaciones TI debe planificar el momento adecuado para la ejecución del despliegue, ya que éste puede afectar algunos servicios e incidir en las operaciones en línea y/o en el procesamiento de datos.
Ejecutar paso	El ingeniero de operaciones TI procede al despliegue automático del software por medio de la herramienta de despliegue continuo, valida la ejecución mediante la revisión de <i>logs</i> e informa a los interesados. Si el despliegue se ejecuta de manera correcta, procede a dar aviso a los analistas de calidad para que informen al usuario y soliciten su validación en ambiente de producción.
Ejecutar vuelta atrás	La ejecución de la vuelta atrás se produce en situaciones en las cuales se presentan errores durante el despliegue de los aplicativos en ambientes productivos o cuando el usuario informa errores en el funcionamiento de éstos. Esta es realizada por el ingeniero de TI responsable de la ejecución del paso.
Informar vuelta atrás	La vuelta atrás debe ser informada a los interesados para que estén enterados de la situación y corrijan la causa de los errores presentados. Además, se debe verificar que el estado de los sistemas y/o procesos, que se encuentran en producción, no se haya visto afectado por la falla del paso.

En el siguiente diagrama se presenta el flujo de actividades realizadas por el equipo del **Área de Operaciones TI**

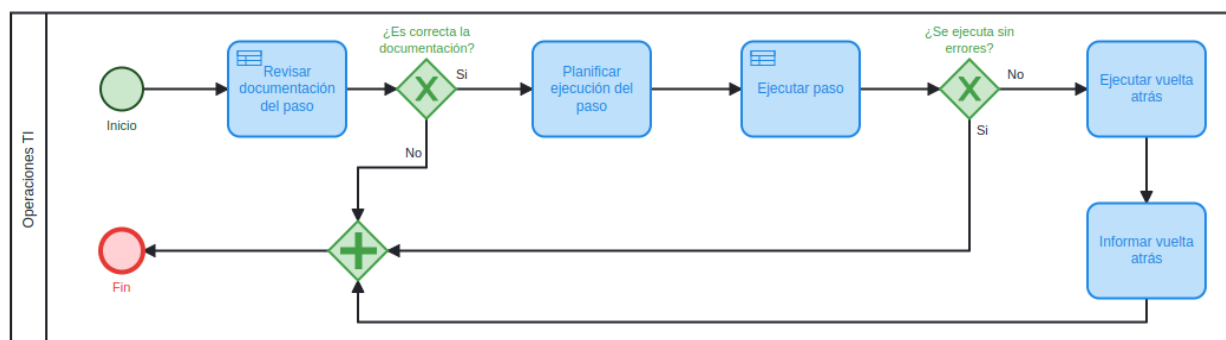


Figura 7. Actividades realizadas por el equipo de Operaciones TI

Operaciones TI	
Entradas	Salidas

• Documento de paso a producción • Solicitud de paso a producción • Solicitud de vuelta atrás si corresponde	• Se revisa documento de paso, se retorna a QA en caso de encontrar observaciones • Se da aviso de la instalación de la aplicación en ambiente de producción • Se ejecuta vuelta atrás en caso de error o solicitud de QA
--	---

La siguiente tabla se representa la matriz de responsabilidades (RECI) corresponde al actual proceso de desarrollo

R = Responsable E = Ejecutor C = Consultado I = Informado

	Actividades/ Roles	Ingeniero de Negocios	Líder Técnico desarrollo	Desarrollador	Analista QA	Ingeniero OTI
1	Definir requerimientos	R/E	C	C		
2	Definir criterios de aceptación	R/E	C	C		
3	Entregar documentación e historias de usuario	R/E	I			
4	Revisar historias de usuario		R/E	C		
5	Definir tiempos y recursos		R/E	C		
6	Construcción		C/I	R/E		
7	Verificar calidad del código		C/I	R/E		
8	Subir códigos a repositorio		I	R/E	I	
9	Desplegar en ambiente de desarrollo		I	R/E		
10	Probar desarrollo		I	R/E	I	
11	Documentar paso		C/I	R/E	I	
12	Solicitar pruebas a QA		I	R/E	I	
13	Planificar pruebas QA			I	R/E	
14	Desplegar en ambiente QA			I	R/E	
15	Ejecutar pruebas		I	I	R/E	
16	Revisar documentación del paso		I	I	R/E	
17	Entregar a producción		I	I	R/E	I
18	Solicitar validación del usuario		I	I	R/E	I
19	Solicitar vuelta atrás		I	I	R/E	I
20	Revisar documentación del paso		I	I	I	R/E
21	Planificar Ejecución del Paso		I	I	I	R/E
22	Ejecutar Paso	I	I	I	I	R/E
23	Ejecutar Vuelta Atrás	I	I	I	I	R/E



	Actividades/ Roles	Ingeniero de Negocios	Líder Técnico desarrollo	Desarrollador	Analista QA	Ingeniero OTI
24	Informar Vuelta Atrás	I	I	I	I	R/E

Anexo 3 Cálculo de niveles de madurez de SAMM

La herramienta SAMM (es decir, cuestionario) consta de 90 preguntas (5 funciones de negocio multiplicadas por 3 prácticas de seguridad, multiplicadas por 2 líneas de acción y multiplicadas por 3 niveles de madurez) que se responden en escalas de 4 puntos entre 0 y 1, donde cero indica que la actividad no está presente y los valores restantes (es decir, 0.25, 0.5 y 1) indican un alcance cada vez más amplio en la adopción de la práctica. Cada práctica recibe una calificación entre 0 y 3 correspondiente a la suma de las respuestas promedio en las dos corrientes para cada nivel de madurez (ver Fórmula (1)).

$$Calificación = \sum_{madurez=1}^3 \frac{(LineaA + LineaB) \cdot madurez}{2} \quad (1)$$

Donde $madurez \in [1, 2, 3]$ and $LineaA|B \in [0, 0.25, 0.50, 1]$

Por lo tanto, una práctica con una calificación de 3 indica que todas las actividades, desde las menos hasta las más maduras, se realizan en el mayor alcance posible.

Las posibles respuestas a las preguntas son:

- No (0)
- Sí, para algunas aplicaciones (0.25)
- Sí, para al menos la mitad de las solicitudes (0.50)
- Sí, para la mayoría o todas las aplicaciones (1)

Por ejemplo, para la práctica Construcción Segura de la función de Implementación, se tienen

Nivel	Pregunta Linea A	Pregunta Linea B	Respuesta A	Respuesta B	Puntaje A	Puntaje B	Promedio
1	¿Se describe formalmente su proceso completo de construcción?	¿Tiene un conocimiento sólido sobre las dependencias en las que confía?	Sí, para algunas aplicaciones	Sí, para al menos la mitad de las solicitudes	0,25	0,5	0,38
2	¿El proceso de construcción está completamente automatizado?	¿Maneja el riesgo de dependencia de terceros mediante un proceso formal?	Sí, para al menos la mitad de las solicitudes	Sí, para al menos la mitad de las solicitudes	0,5	0,5	0,5
3	¿Hace cumplir controles de seguridad automatizados en sus procesos de compilación?	¿Evita la compilación de software si se ve afectado por vulnerabilidades en las dependencias?	Sí, para al menos la mitad de las solicitudes	Sí, para al menos la mitad de las solicitudes	0,5	0,5	0,5
	Nivel de madurez						1,38

El nivel de madurez de la función de negocio corresponde al valor promedio de los niveles de madurez de las prácticas de seguridad contenidas en la función.

Para hacer una evaluación de madurez se puede utilizar la caja de herramientas de SAMM, lo que facilita ésta labor, y se encuentra disponible en <https://owasp samm.org/assessment/>

Anexo 4 Descripción de actividades de la metodología propuesta (Se destacan las actividades agregadas)

En esta sección se detallan las actividades que componen la nueva metodología propuesta para desarrollo de software seguro.

Se entregan:

- Se entrega el listado de actividades
- Diagramas
- Entradas y salidas
- Matriz de responsabilidades

Procesos y Negocios, Desarrollo y QA

Actividad	Descripción	Actividad del modelo SAMM relacionada	Requisito PCI relacionado
3.4.1 Definir requerimientos funcionales	Se deben especificar, de manera clara y concisa, los servicios que prestará el software y la forma en que éste tiene que reaccionar ante determinadas entradas. Cuando se habla de entradas, no necesariamente se refiere sólo a las entradas de los usuarios. Pueden ser interacciones con otros sistemas, respuestas automáticas, procesos predefinidos, etc. También se debe establecer de forma explícita lo que el sistema no debe hacer, siempre y cuando el resultado de su comportamiento sea una respuesta funcional al usuario u otro sistema.		
3.4.2 Evaluar el riesgo de la aplicación	Se debe utilizar un método sencillo para valorar los riesgos de la aplicación/servicio. Los riesgos deben ser evaluados, estimando el impacto potencial que supone para el negocio de la organización un ataque de: Violación de la confidencialidad Violación de la integridad Disponibilidad de la aplicación. Para ello, se puede utilizar un conjunto reducido de preguntas (5 a 10) para comprender las características más importantes de la aplicación: p.ej. si la aplicación procesa datos financieros, si está expuesta a Internet o si se trata de datos personales, etc. Se debe clasificar el nivel de riesgo de las aplicaciones siguiendo un método cualitativo simple del tipo: alto/medio/bajo. Es importante utilizar estos valores para representar y comparar el riesgo de las diferentes aplicaciones entre sí.	Las actividades 3.4.2 y 3.4.3 se relacionan con la práctica de Evaluación de Amenazas (TA1) del modelo OWASP SAMM, que tiene por objetivo tener en cuenta de forma explícita la seguridad durante el proceso de creación de requisitos del software.	Las actividades 3.4.2 y 3.4.3 ayudan a mejorar el cumplimiento de los siguientes requisitos de las normas PCI: 6. Desarrollar y mantener sistemas y aplicaciones seguros. 6.1 Establecer un proceso para identificar las vulnerabilidades de seguridad por medio de fuentes externas conocidas para obtener información sobre las vulnerabilidades de seguridad, y asignar una clasificación de riesgo (por ejemplo, "alto", "medio" o "bajo") a las vulnerabilidades de seguridad recientemente descubiertas. 12. Mantener una política que aborde la seguridad de la información para todo el personal. 12.2 Implementar un proceso de evaluación de riesgos que cumpla con lo siguiente: Se realiza, al menos, una vez al año y después de

Actividad	Descripción	Actividad del modelo SAMM relacionada	Requisito PCI relacionado
			implementar cambios significativos en el entorno (por ejemplo, adquisiciones, fusiones o reubicaciones, etc.). Identifica activos críticos, amenazas y vulnerabilidades. Entrega de resultados en un análisis formal y documentado de riesgo.
3.4.3 Realizar modelado básico de amenazas para la aplicación	Se debe identificar, evaluar y gestionar las amenazas del sistema, los defectos de diseño en la arquitectura del mismo y las mitigaciones de seguridad recomendadas. Para esta actividad se recomienda el uso de una herramienta que permita realizar el modelado de manera gráfica. El modelado de amenazas es un ejercicio de equipo, que debe incluir a propietarios de productos, arquitectos, encargados de seguridad y analistas de calidad. Es importante implicar al equipo de trabajo y al resto de las partes interesadas en esta actividad con el fin de incrementar la concientización en materia de seguridad y crear una visión compartida sobre la seguridad del sistema. Durante esta actividad se deben utilizar listas de comprobación de amenazas sencillas y relevantes, evitando el uso de modelos complejos que puedan originar reuniones y discusiones extensas.	Las actividades 3.4.2 y 3.4.3 se relacionan con la práctica de Evaluación de Amenazas (TA1) del modelo OWASP SAMM, que tiene por objetivo tener en cuenta de forma explícita la seguridad durante el proceso de creación de requisitos del software.	Las actividades 3.4.2 y 3.4.3 ayudan a mejorar el cumplimiento de los siguientes requisitos de las normas PCI: 6. Desarrollar y mantener sistemas y aplicaciones seguros. 6.1 Establecer un proceso para identificar las vulnerabilidades de seguridad por medio de fuentes externas conocidas para obtener información sobre las vulnerabilidades de seguridad, y asignar una clasificación de riesgo (por ejemplo, “alto”, “medio” o “bajo”) a las vulnerabilidades de seguridad recientemente descubiertas. 12. Mantener una política que aborde la seguridad de la información para todo el personal. 12.2 Implementar un proceso de evaluación de riesgos que cumpla con lo siguiente: Se realiza, al menos, una vez al año y después de implementar cambios significativos en el entorno (por ejemplo, adquisiciones, fusiones o reubicaciones, etc.). Identifica activos críticos, amenazas y vulnerabilidades. Entrega de resultados en un análisis formal y documentado de riesgo.
3.4.4 Identificar requisitos de seguridad	Se deben identificar de manera específica los requisitos de seguridad del software a partir de los requisitos funcionales del proyecto. Se debe analizar las dimensiones de confidencialidad, integridad y disponibilidad deseadas para la aplicación o los datos manejados por ésta y establecer los objetivos de seguridad. Además de lo anterior, para la identificación de los requisitos de seguridad, es preciso tener en cuenta lo establecido por los estándares, normativas y obligaciones que la organización debe cumplir. Se debe tener presente que estas políticas y estándares establecen una línea base de requerimientos de seguridad para todas las aplicaciones y, por lo tanto, constituyen otra fuente de	La actividad 3.4.4 se relaciona con la Práctica Requisitos de Seguridad (SR1) del modelo OWASP SAMM, que tiene por objetivo conseguir que se consideren de forma específica los requisitos de seguridad durante la fase de captura de requisitos del sistema.	La actividad 3.4.4 ayuda a mejorar el cumplimiento de los siguientes requisitos de las normas PCI: 6. Desarrollar y mantener sistemas y aplicaciones seguros. 6.3 Desarrolle aplicaciones de software internas y externas (incluso acceso administrativo a aplicaciones basadas en la web) de manera segura. 6.6 En el caso de aplicaciones web públicas, se deben tratar las nuevas amenazas y

Actividad	Descripción	Actividad del modelo SAMM relacionada	Requisito PCI relacionado
	requisitos de seguridad que deben ser considerados. Por otro lado, se tiene que analizar la funcionalidad de la aplicación o servicio desde la perspectiva de un atacante, con el fin de comprender cómo ésta podría ser utilizada por un usuario malintencionado para intentar realizar un ataque. Este análisis facilita la identificación de requisitos de protección adicionales que darán lugar a nuevos requisitos de seguridad. En la sección "Anexo 1 Requisitos de seguridad extraídos desde las normas PCI DSS" del Anexo se presenta un listado de requisitos de seguridad exigidos por la normativa PCI DSS, algunos de los cuales pueden ser aplicados a todo tipo de software.		vulnerabilidades continuamente, asegurándose de que estas aplicaciones se protejan contra ataques conocidos.
3.4.5 Integrar en el proceso los requisitos de seguridad	Los requisitos de seguridad deben ser gestionados de la misma manera que los requisitos funcionales. Para ello es necesario incluirlos en los documentos de análisis de requisitos, creación de historias de usuario (si se trabaja con metodología ágil), etc. Es decir, proceder de la misma forma que con los requisitos funcionales. Independientemente de la clasificación de los requisitos de seguridad, se deben seguir las buenas prácticas para redactarlos. Es decir, deben ser específicos, medibles, alcanzables, razonables y limitados en el tiempo. Al igual que sucede con los requisitos funcionales de las aplicaciones y/o servicios, los requisitos de seguridad deben ser incorporados durante el desarrollo del producto. Por lo tanto, deben ser analizados, documentados, asignados, priorizados, probados, y validados. Los requisitos de seguridad, en definitiva, especifican características del producto final y deben ser tratados y gestionados como tal por parte de los equipos de desarrollo.	La actividad 3.4.5 se relaciona con la Práctica Requisitos de Seguridad (SR2) del modelo OWASP SAMM, cuyo objetivo es aumentar la granularidad y el detalle de los requisitos de seguridad, y derivarlos a partir de la lógica de negocio de la aplicación y de los riesgos conocidos.	Las actividades 3.4.5 y 3.4.6 ayudan a mejorar el cumplimiento de los siguientes requisitos de las normas PCI: 6. Desarrollar y mantener sistemas y aplicaciones seguros. 6.7 Asegúrese de que las políticas de seguridad y los procedimientos operativos para desarrollar y mantener seguros los sistemas y las aplicaciones estén documentados, implementados y que sean de conocimiento para todas las partes afectadas.
3.4.6 Definir criterios de aceptación funcionales y de seguridad	Dentro de los criterios de aceptación se debe incorporar escenarios relacionados no sólo con los requisitos funcionales, sino también con el comportamiento de la aplicación frente a requisitos de seguridad. Los criterios de aceptación, relacionados con la seguridad, deben establecer si los controles funcionan según lo previsto y satisfacen los requisitos del proyecto. Para ello, se tienen que definir pruebas positivas y negativas. Las primeras verifican que los controles de seguridad de la aplicación satisfacen los requisitos de seguridad establecidos y validan su correcto funcionamiento. Estos requisitos suelen ser de naturaleza funcional. Por su parte, las pruebas negativas se ocupan para validar la calidad de los controles de	La actividad 3.4.6 se relaciona con la Práctica de Pruebas Orientadas a Requisitos (RT1) del modelo OWASP SAMM, cuyo objetivo es encontrar vulnerabilidades básicas y otros problemas de seguridad de forma oportuna.	Las actividades 3.4.5 y 3.4.6 ayudan a mejorar el cumplimiento de los siguientes requisitos de las normas PCI: 6. Desarrollar y mantener sistemas y aplicaciones seguros. 6.7 Asegúrese de que las políticas de seguridad y los procedimientos operativos para desarrollar y mantener seguros los sistemas y las aplicaciones estén documentados, implementados y que sean de conocimiento para todas las partes afectadas.

Actividad	Descripción	Actividad del modelo SAMM relacionada	Requisito PCI relacionado
	seguridad y tienen como objetivo detectar fallas de implementación y diseño mediante pruebas de uso y abuso. La finalidad de las pruebas a establecer durante esta etapa es verificar la eficacia de los controles de seguridad típicos. Por lo cual, es necesario documentar los procedimientos para validar que los controles de seguridad básicos funcionan tal y como es esperado. A grandes rasgos, esto supone verificar el correcto funcionamiento de los controles de confidencialidad, integridad, disponibilidad de datos y servicio. Las pruebas básicas de seguridad deben incluir, como mínimo, pruebas de autenticación, control de acceso, validación de datos de entrada, controles de cifrado y codificación de datos.		
3.4.7 Entregar documentación e historias de usuario (incluir requisitos funcionales y de seguridad)	Como ya se ha mencionado, los requisitos de seguridad deben ser gestionados de la misma forma que se hace con los requisitos funcionales. Por lo tanto, éstos deben ser documentados e incluidos dentro de las historias de usuario que son entregadas a los equipos de desarrollo para su evaluación, priorización, estimación de esfuerzos, asignación, etc. De la misma manera que se hace con los requisitos funcionales. Además, se tiene que incorporar dentro de la documentación los criterios de aceptación definidos, de modo que éstos antecedentes sean tomados en cuenta por el equipo de desarrollo en la construcción de las aplicaciones.	La actividad 3.4.7 se relaciona con la Práctica Requisitos de Seguridad (SR2) del modelo OWASP SAMM, cuyo objetivo es aumentar la granularidad y el detalle de los requisitos de seguridad, y derivarlos a partir de la lógica de negocio de la aplicación y de los riesgos conocidos.	

En el siguiente diagrama se presenta el flujo de actividades realizadas por los equipos de **Procesos y Negocios, Desarrollo y QA**.

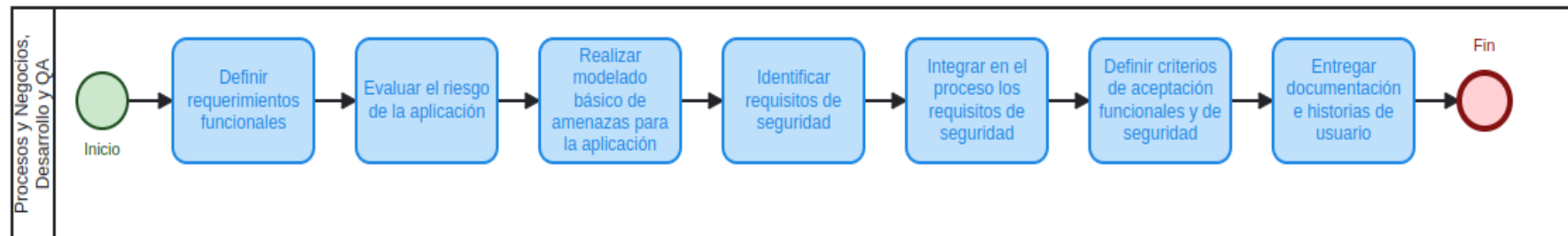


Figura 8. Actividades realizadas por el equipo de Procesos y Negocios, Desarrollo y QA

Entradas	Salidas
Ideas o requerimientos entregadas al área por los usuarios	Se entregan documentos estructurados a Desarrollo con el detalle de los requerimientos, conocidas como historias de usuario. En estos documentos se incluyen los requerimientos de seguridad para la aplicación.

Desarrollo

Actividad	Descripción	Actividad del modelo SAMM relacionada	Requisito PCI relacionado
3.4.8 Revisar historias de usuario	Se debe ejecutar una revisión de las historias de usuario y de los criterios de aceptación para validar su consistencia con los requisitos funcionales y de seguridad. Además, se debe verificar que dentro de los antecedentes se consideran requisitos de seguridad consistentes con las normas y estándares exigidos a la organización.		
3.4.9 Evaluar la arquitectura de la aplicación	Se debe evaluar la arquitectura de alto nivel de la aplicación y establecer si ésta cumple con las medidas de seguridad básicas. Para ello, se debe crear una visión de la arquitectura global de la aplicación y examinar para validar la provisión de mecanismos de seguridad generales como la autenticación, la autorización, la gestión de usuarios y derechos, la comunicación segura, la protección de datos, la privacidad, la gestión de claves, la gestión de registros, etc. De manera general, en la arquitectura se puede identificar cualquier funcionalidad relacionada con la seguridad y evaluar la correcta provisión de los mecanismos de control de forma ad hoc, desde las perspectivas de los usuarios anónimos, los usuarios autorizados y los roles específicos de la aplicación. El examen se basa en los artefactos del proyecto, documentos de arquitectura o diseño, o entrevistas con responsables y el personal técnico. Se deberán considerar todos los elementos de la infraestructura, incluyendo aquellos sistemas, componentes y bibliotecas (incluidos los SDK) que no son específicos de la aplicación, pero que proporcionan apoyo directo para utilizar o gestionar las aplicaciones en la organización.	La actividad 3.4.9 se relaciona con la Práctica de Evaluación de la Arquitectura (AA1) del modelo OWASP SAMM, cuyo objetivo es revisar la arquitectura de las aplicaciones para asegurar que se han implementado mitigaciones para los riesgos de seguridad conocidos.	La actividad 3.4.9 ayuda a mejorar el cumplimiento de los siguientes requisitos de las normas PCI: 2. No usar los valores predeterminados suministrados por el proveedor para las contraseñas del sistema y otros parámetros de seguridad. 2.4 Lleve un inventario de los componentes del sistema que están dentro del alcance de las PCI DSS. 6. Desarrollar y mantener sistemas y aplicaciones seguros. 6.3 Desarrolle aplicaciones de software internas y externas (incluso acceso administrativo a aplicaciones basadas en la web) de manera segura. 6.6 En el caso de aplicaciones web públicas, se deben tratar las nuevas amenazas y vulnerabilidades continuamente, asegurándose de que estas aplicaciones se protejan contra ataques conocidos.
3.4.10 Identificar y evaluar herramientas y tecnologías	El objetivo de esta actividad es tener una visión clara de las herramientas y tecnologías empleadas en el desarrollo y crear conciencia de cómo estas pueden introducir riesgos de seguridad en el producto final. En la actualidad, en el desarrollo de software es bastante frecuente	La actividad 3.4.10 se relaciona con las Prácticas de Arquitectura de Seguridad (SA1) y Construcción Segura (SB1) del modelo OWASP SAMM, cuyo objetivo es insertar en el proceso de desarrollo de software el hábito de tener en cuenta la seguridad en la fase de diseño del software.	La actividad 3.4.10 ayuda a mejorar el cumplimiento de los siguientes requisitos de las normas PCI: 6. Desarrollar y mantener sistemas y aplicaciones seguros.

Actividad	Descripción	Actividad del modelo SAMM relacionada	Requisito PCI relacionado
	hacer uso de nuevas tecnologías, frameworks, utilidades y herramientas con la finalidad de facilitar y/o acelerar el proceso de desarrollo o mejorar aspectos como la versatilidad y la escalabilidad del producto final. No hay nada malo en utilizar nuevas tecnologías para mejorar las capacidades de procesos y productos, pero es preciso tener en cuenta que se pueden introducir nuevos riesgos para la organización que hay que gestionar adecuadamente. Para evitar asumir nuevos riesgos de forma inconsciente, se deben identificar las tecnologías, frameworks y herramientas más importantes que se utilizan para la aplicación, incluyendo los entornos de desarrollo y operativo. Se tiene que realizar un análisis de las características y del estado de la seguridad de estos componentes e informar sobre los hallazgos o defectos de seguridad relevantes para que puedan ser gestionados adecuadamente.		6.2 Se debe asegurar que todos los software y componentes del sistema tengan instalados parches de seguridad proporcionados por los proveedores que ofrecen protección contra vulnerabilidades conocidas. Instalar los parches importantes de seguridad dentro de un plazo de un mes de su lanzamiento. 6.3 Desarrolle aplicaciones de software internas y externas (incluso acceso administrativo a aplicaciones basadas en web) de manera segura. 6.6 En el caso de aplicaciones web públicas, trate las nuevas amenazas y vulnerabilidades continuamente y asegúrese de que estas aplicaciones se protejan contra ataques conocidos.
3.4.11 Definir tiempos y recursos	Una vez hechas las evaluaciones correspondientes y aceptadas las historias de usuario se procede a acordar los plazos y los recursos necesarios para su construcción. Las historias que requieren mayor esfuerzo se pueden dividir en tareas que son distribuidas entre los miembros de la célula y los plazos se consideran desde el inicio de la construcción hasta el momento de la entrega al ambiente de pruebas de calidad (QA). Dentro de los plazos se deben considerar todas aquellas tareas que tienen relación con los requisitos de seguridad identificados para la aplicación. Se tiene que tomar en cuenta que el equipo de desarrollo debe participar en actividades relacionadas con evaluación de arquitectura, evaluación de amenazas, evaluación de tecnologías, etc.		
3.4.12 Construcción	Durante la etapa de construcción el desarrollador procede a la creación de los componentes de software necesarios para cumplir con las historias de usuario. Para ello hace uso de un IDE (Entorno de Desarrollo Integrado) y de librerías que forman parte del stack tecnológico de la empresa. A medida que la construcción del software avanza, el desarrollador ejecuta pruebas unitarias sobre los componentes que va creando. Además, debe verificar de manera regular la calidad del código que va creando. Por lo general, durante esta etapa surgen algunas dudas, que el desarrollador puede aclarar directamente con el ingeniero de negocios y/o el usuario.		
3.4.13 Actualizar	A medida que el proyecto avanza, el equipo de desarrollo debe	La actividad 3.4.13 se relaciona con la Práctica de	La actividad 3.4.13 ayuda a mejorar el



Actividad	Descripción	Actividad del modelo SAMM relacionada	Requisito PCI relacionado
inventario de componentes	actualizar el inventario de componentes que forman la aplicación/servicio. Esto tiene como finalidad tener un control sobre el desarrollo y que los miembros del equipo tengan a su disposición documentación suficiente para futuras actualizaciones o corrección de incidencias. Por otro lado, esto permitiría facilitar el proceso de desmantelamiento del sistema cuando éste deje de operar. Existen múltiples razones por las que se puede llegar a tener software no controlado: Entornos de prueba que no se gestionan con el mismo rigor que los productivos. Migraciones o actualizaciones de productos. Cambios de equipamiento. Pruebas temporales de productos. Diseño no óptimo de las autorizaciones en los sistemas. Pruebas de concepto para el desarrollo de software.	Gestión Operativa (OM1) del modelo OWASP SAMM, cuyo objetivo es comenzar a realizar actividades básicas de gestión, algunas de las cuales sirven para conocer el estado actual y los componentes de los sistemas en ambientes productivos.	cumplimiento de los siguientes requisitos de las normas PCI: 2. No usar los valores predeterminados suministrados por el proveedor para las contraseñas del sistema y otros parámetros de seguridad. 2.4 Lleve un inventario de los componentes del sistema que están dentro del alcance de las PCI DSS. 6. Desarrollar y mantener sistemas y aplicaciones seguros. 6.4 Siga los procesos y procedimientos de control de todos los cambios en los componentes del sistema.
3.4.14 Verificar calidad del código	El objetivo de la etapa es evaluar el código para detectar errores, fallas y normas de calidad establecidas para la organización. Por otro lado, el proceso de revisión de código trae como beneficio intangible una mejora de las aptitudes de codificación de los equipos. Para esta actividad se propone efectuar una revisión por pares, lo que corresponde a una técnica en la cual dos o más miembros del equipo se encargan de revisar el código fuente para verificar que éste se encuentre acorde a las normas establecidas, identificar errores en la lógica y proponer mejoras o correcciones. Por otro lado, cada desarrollador debe verificar la calidad del código construido por medio de un escáner que se encuentre integrado al IDE utilizado. Esto tiene como finalidad que el código se apegue a las mejores prácticas y que se encuentre libre de errores o vulnerabilidades conocidas.	La actividad 3.4.14 se relaciona con la Práctica de Pruebas de Seguridad (ST1) del modelo OWASP SAMM, cuyo objetivo es realizar pruebas de seguridad, tanto manuales como automatizadas, para descubrir defectos de seguridad.	La actividad 3.4.14 ayuda a mejorar el cumplimiento de los siguientes requisitos de las normas PCI: 6. Desarrollar y mantener sistemas y aplicaciones seguros. 6.5 Aborde las vulnerabilidades de codificación comunes en los procesos de desarrollo de software.
3.4.15 Subir códigos a repositorio	Los códigos que pasan los criterios de calidad deben ser almacenados en un repositorio central que permita su versionamiento. Este es un requisito que se debe cumplir para que los productos desarrollados puedan ser desplegados en los respectivos ambientes, ya que la herramienta de despliegue continuo construye los aplicativos ejecutables a partir de los códigos fuente presentes en el repositorio.	Las actividades 3.4.15 y 3.4.16 se relacionan con la Práctica de Construcción Segura (SB1) del modelo OWASP SAMM, cuyo objetivo es conseguir que el proceso de construcción de las aplicaciones sea repetible y consistente.	Las actividades 3.4.15 y 3.4.16 ayudan a mejorar el cumplimiento de los siguientes requisitos de las normas PCI: 6. Desarrollar y mantener sistemas y aplicaciones seguros. 6.3 Desarrolle aplicaciones de software internas y externas (incluso acceso administrativo a aplicaciones basadas en web) de manera segura.
3.4.16 Desplegar en ambiente de desarrollo	Para verificar que el aplicativo se desplegará correctamente en los ambientes de calidad y producción, los desarrolladores deben	Las actividades 3.4.15 y 3.4.16 se relacionan con la Práctica de Construcción Segura (SB1) del	Las actividades 3.4.15 y 3.4.16 ayudan a mejorar el cumplimiento de los siguientes requisitos de las



Actividad	Descripción	Actividad del modelo SAMM relacionada	Requisito PCI relacionado
	realizar una prueba de despliegue en ambiente de desarrollo por medio de la herramienta de despliegue continuo. Durante este proceso nuevamente se revisa la calidad del código, ya que la herramienta tiene que tener definida una "Quality Gate" asociada a un escáner que no permite desplegar aplicativos cuyos códigos no cumplen con los criterios de buenas prácticas, además de estar libres de vulnerabilidades.	modelo OWASP SAMM, cuyo objetivo es conseguir que el proceso de construcción de las aplicaciones sea repetible y consistente.	normas PCI: 6. Desarrollar y mantener sistemas y aplicaciones seguros. 6.3 Desarrolle aplicaciones de software internas y externas (incluso acceso administrativo a aplicaciones basadas en web) de manera segura.
3.4.17 Probar requisitos funcionales	Los desarrolladores tienen que ejecutar pruebas funcionales del software. Las pruebas deben estar acorde con los requisitos definidos en la etapa de definición de criterios de aceptación. Esto tiene por finalidad verificar que se cumple con los requisitos funcionales antes de solicitar la ejecución de pruebas en ambiente de calidad. Una prueba funcional es una prueba de tipo caja negra basada en la ejecución, revisión y retroalimentación de las funcionalidades previamente diseñadas para el software.	Las actividades 3.4.17 y 3.4.18 se relacionan con la Práctica Pruebas Orientadas a Requisitos (RT1) del modelo OWASP SAMM, cuyo objetivo es encontrar vulnerabilidades básicas y otros problemas de seguridad de manera oportuna.	
3.4.18 Probar requisitos no funcionales	Los desarrolladores tienen que ejecutar pruebas relacionadas con requisitos no funcionales del software. Las pruebas deben estar acorde con los requisitos definidos en la etapa de definición de criterios de aceptación. Esto tiene por finalidad verificar que se cumple con estos requisitos antes de solicitar la ejecución de pruebas en ambiente de calidad. Dependiendo de las características del software desarrollado se deben ejecutar Pruebas de carga Pruebas de rendimiento Pruebas de estrés	Las actividades 3.4.17 y 3.4.18 se relacionan con la Práctica Pruebas Orientadas a Requisitos (RT1) del modelo OWASP SAMM, cuyo objetivo es encontrar vulnerabilidades básicas y otros problemas de seguridad de manera oportuna.	
3.4.19 Ejecutar pruebas de seguridad	Las pruebas de seguridad de las aplicaciones se pueden llevar a cabo mediante la inspección del código fuente de una aplicación sin ejecutarla (análisis estático), u observando el comportamiento de la aplicación en respuesta a diversas condiciones de entrada (análisis dinámico). Durante esta etapa se deben utilizar herramientas automatizadas de pruebas de seguridad. Esto permite realizar las pruebas de manera más eficiente y obtener resultados más consistentes y de mayor calidad. La organización debe seleccionar las herramientas adecuadas en función de varios factores, como la profundidad y la precisión de la inspección, la solidez y la precisión de los casos de prueba de seguridad, las integraciones disponibles con otras herramientas, el modelo de uso, el costo, etc.	Las actividades 3.4.19 se relacionan con la Práctica Pruebas de Seguridad (ST1) del modelo OWASP SAMM, cuyo objetivo es realizar pruebas de seguridad, tanto manuales como automatizadas, para descubrir defectos de seguridad.	La actividad 3.4.19 ayuda a mejorar el cumplimiento de los siguientes requisitos de las normas PCI: 6. Desarrollar y mantener sistemas y aplicaciones seguros. 6.3 Desarrolle aplicaciones de software internas y externas (incluso acceso administrativo a aplicaciones basadas en web) de manera segura.



Actividad	Descripción	Actividad del modelo SAMM relacionada	Requisito PCI relacionado
	Por otro lado, se deben realizar pruebas de seguridad manuales selectivas (se pueden emplear previamente una combinación de herramientas de análisis estático y dinámico para guiar o enfocar la revisión manual), con el fin de analizar más a fondo ciertas partes de la aplicación, procediendo como lo haría un atacante. Se deben probar las funcionalidades de alto riesgo, tales como: módulos de autenticación, los puntos de control de acceso, la gestión de sesiones, interfaces externas y validación de datos de entrada.		
3.4.20 Documentar paso	Una vez que el desarrollador finaliza sus pruebas satisfactoriamente, se elabora la documentación necesaria para el despliegue del nuevo aplicativo en los ambientes de pruebas de calidad (QA y Testing) y producción. Esta documentación debe contener los pasos necesarios para la instalación de la aplicación, así como para ejecutar la vuelta atrás en caso de que sea necesario.	La actividad 3.4.20 se relaciona con la Práctica de Despliegue Seguro (SD1) del modelo OWASP SAMM, cuyo objetivo es que el procedimiento para el despliegue de las aplicaciones se encuentre correctamente documentado.	La actividad 3.4.20 ayuda a mejorar el cumplimiento de los siguientes requisitos de las normas PCI: 6. Desarrollar y mantener sistemas y aplicaciones seguros. 6.3 Desarrolle aplicaciones de software internas y externas (incluso acceso administrativo a aplicaciones basadas en web) de manera segura. 6.7 Asegúrese de que las políticas de seguridad y los procedimientos operativos para desarrollar y mantener seguros los sistemas y las aplicaciones estén documentados, implementados y que sean de conocimiento para todas las partes afectadas.
3.4.21 Solicitar pruebas a QA	La documentación del paso se sube a la plataforma de gestión de proyectos, se informa la entrega al área de QA y se asigna a él o los analistas correspondientes para la planificación de las pruebas.		

En el siguiente diagrama se presenta el flujo de actividades realizadas por el equipo de **Desarrollo**

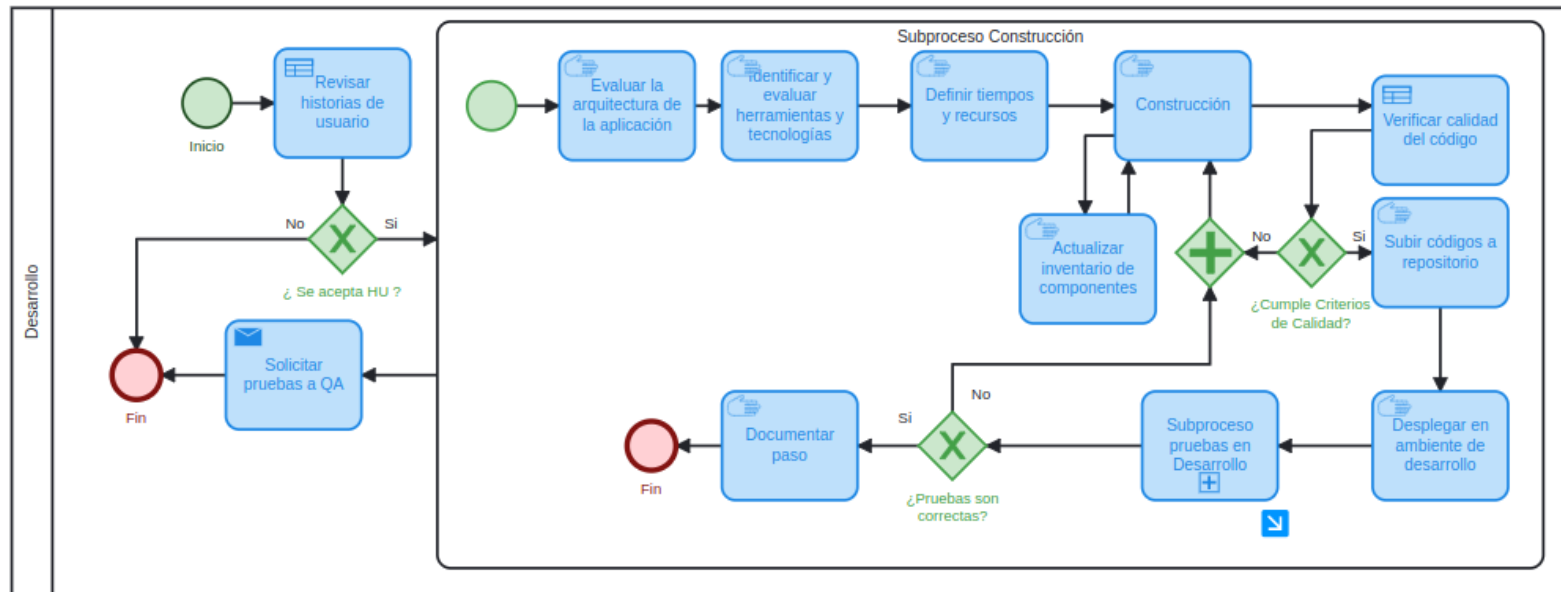


Figura 9. Actividades realizadas por el equipo de Desarrollo

La siguiente figura representa al subproceso “Pruebas en Desarrollo”

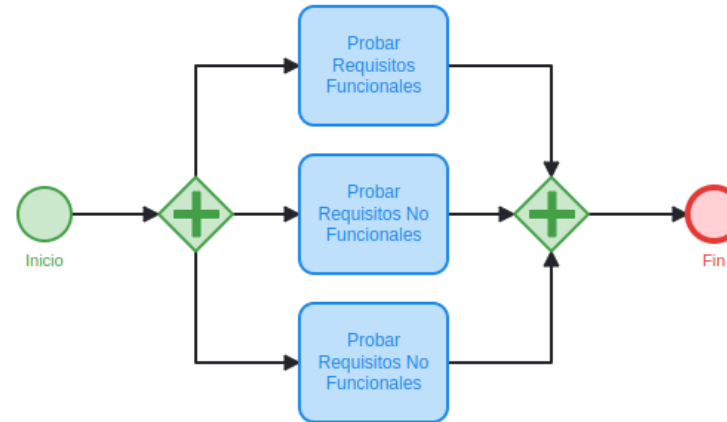


Figura 10. subprocesso “Pruebas en ambiente de Desarrollo”

Desarrollo	
Entradas	Salidas
Historias de usuario entregadas por al área de Procesos y Negocios que incluye los requerimientos de seguridad	- Revisan las HU, en caso de identificar inconsistencias se devuelve a Procesos y Negocios - Se entregan los códigos fuente de la aplicación, el documento de paso a ambiente de producción a QA y se solicitan las pruebas en ambiente de QA

QA y Testing

Actividad	Descripción	Actividad del modelo SAMM relacionada	Requisito PCI relacionado
3.4.22 Planificar pruebas QA	El o los analistas de calidad de software asignados a la célula reciben las solicitudes y deben planificar los recursos necesarios para la ejecución de las pruebas en ambiente de calidad. La planificación debe considerar pruebas funcionales, no funcionales y de seguridad.		
3.4.23 Desplegar en ambiente QA	Antes de proceder con la ejecución de las pruebas de calidad, el software desarrollado tiene que ser instalado en el ambiente correspondiente por medio de la herramienta de despliegue continuo. Se requiere que esta actividad sea ejecutada por los analistas de calidad, quienes deben informar cualquier error que se	La actividad 3.4.23 se relaciona con la Práctica de Construcción Segura (SB1) del modelo OWASP SAMM, cuyo objetivo es conseguir que el proceso de construcción de las aplicaciones sea repetible y consistente.	



Actividad	Descripción	Actividad del modelo SAMM relacionada	Requisito PCI relacionado
	presente. De este modo, se realiza una segunda prueba de despliegue del software, esta vez en el ambiente de calidad (anteriormente en ambiente de desarrollo), lo que permite verificar la construcción y despliegue de éste.		
3.4.24 Probar requisitos funcionales	Los analistas tienen que ejecutar pruebas funcionales del software. Las pruebas deben estar acorde con los requisitos definidos en la etapa de definición de criterios de aceptación. Esto tiene por finalidad verificar que se cumple con los requisitos funcionales antes de solicitar la instalación del software en ambiente productivo.	Las actividades 3.4.24 y 3.4.25 se relacionan con la Práctica Pruebas Orientadas a Requisitos (RT1) del modelo OWASP SAMM, cuyo objetivo es encontrar vulnerabilidades básicas y otros problemas de seguridad de manera oportuna.	
3.4.25 Probar requisitos no funcionales	Los analistas tienen que ejecutar pruebas relacionadas con requisitos no funcionales del software. Las pruebas deben estar acorde con los requisitos definidos en la etapa de definición de criterios de aceptación. Esto tiene por finalidad verificar que se cumple con estos requisitos antes de solicitar la instalación del software en ambiente productivo. Dependiendo de las características del software desarrollado se deben ejecutar Pruebas de carga Pruebas de rendimiento Pruebas de estrés	Las actividades 3.4.24 y 3.4.25 se relacionan con la Práctica Pruebas Orientadas a Requisitos (RT1) del modelo OWASP SAMM, cuyo objetivo es encontrar vulnerabilidades básicas y otros problemas de seguridad de manera oportuna.	
3.4.26 Ejecutar pruebas de seguridad	Al igual que en la actividad 3.4.19 los analistas de calidad deben ejecutar pruebas de seguridad, esta vez con mayor rigurosidad que las hechas por los desarrolladores para, de esta manera, aplicar una segunda revisión que garantice que el software cumple con los requisitos de seguridad establecidos. Para esta actividad el analista se debe apoyar en herramientas automatizadas que le permitan acelerar optimizar el tiempo necesario para las pruebas.	Las actividades 3.4.26 se relacionan con la Práctica Pruebas de Seguridad (ST1) del modelo OWASP SAMM, cuyo objetivo es realizar pruebas de seguridad, tanto manuales como automatizadas, para descubrir defectos de seguridad.	La actividad 3.4.26 ayuda a mejorar el cumplimiento de los siguientes requisitos de las normas PCI: 6. Desarrollar y mantener sistemas y aplicaciones seguros. 6.3 Desarrolle aplicaciones de software internas y externas (incluso acceso administrativo a aplicaciones basado en web) de manera segura
3.4.27 Revisar y corregir documentación del paso	Una vez que se han recopilado las evidencias sobre el correcto funcionamiento del software y se tiene la conformidad del usuario con las pruebas, los analistas de calidad tienen que hacer una nueva revisión de la documentación de paso a producción. En caso de que ésta presente alguna falencia, se tiene que informar al equipo de desarrollo para que la documentación sea corregida.	La actividad 3.4.27 se relaciona con la Práctica de Despliegue Seguro (SD1) del modelo OWASP SAMM, cuyo objetivo es que el procedimiento para el despliegue de las aplicaciones se encuentre correctamente documentado.	La actividad 3.4.27 ayuda a mejorar el cumplimiento de los siguientes requisitos de las normas PCI: 6. Desarrollar y mantener sistemas y aplicaciones seguros. 6.7 Asegúrese de que las políticas de seguridad y los procedimientos operativos para desarrollar y mantener seguros los sistemas y las aplicaciones estén documentados, implementados y que sean de conocimiento para todas las partes afectadas.

Actividad	Descripción	Actividad del modelo SAMM relacionada	Requisito PCI relacionado
3.4.28 Entregar a producción	Una vez que la documentación se encuentra verificada, se procede a enviar la solicitud de paso a producción al área de Operaciones TI, que es la encargada del despliegue de los aplicativos en los ambientes productivos.		
3.4.29 Solicitar validación del usuario	Una vez que el paso a ambiente productivo se ha ejecutado, el analista de calidad solicita al usuario correspondiente que haga una validación de las funcionalidades del software entregado. Si éste confirma el correcto funcionamiento, se da aviso a operaciones TI para que dé por finalizado el paso de manera correcta. En caso contrario, el analista de calidad informa a operaciones TI que se debe ejecutar la vuelta atrás del paso e informar éste como fallido.		
3.4.30 Solicitar vuelta atrás	En caso de que el usuario indique errores al hacer sus validaciones en ambiente de producción se solicita la ejecución de vuelta atrás del paso a producción al área de Operaciones TI.		

En el siguiente diagrama se presenta el flujo de actividades realizadas por el equipo de **QA y Testing**

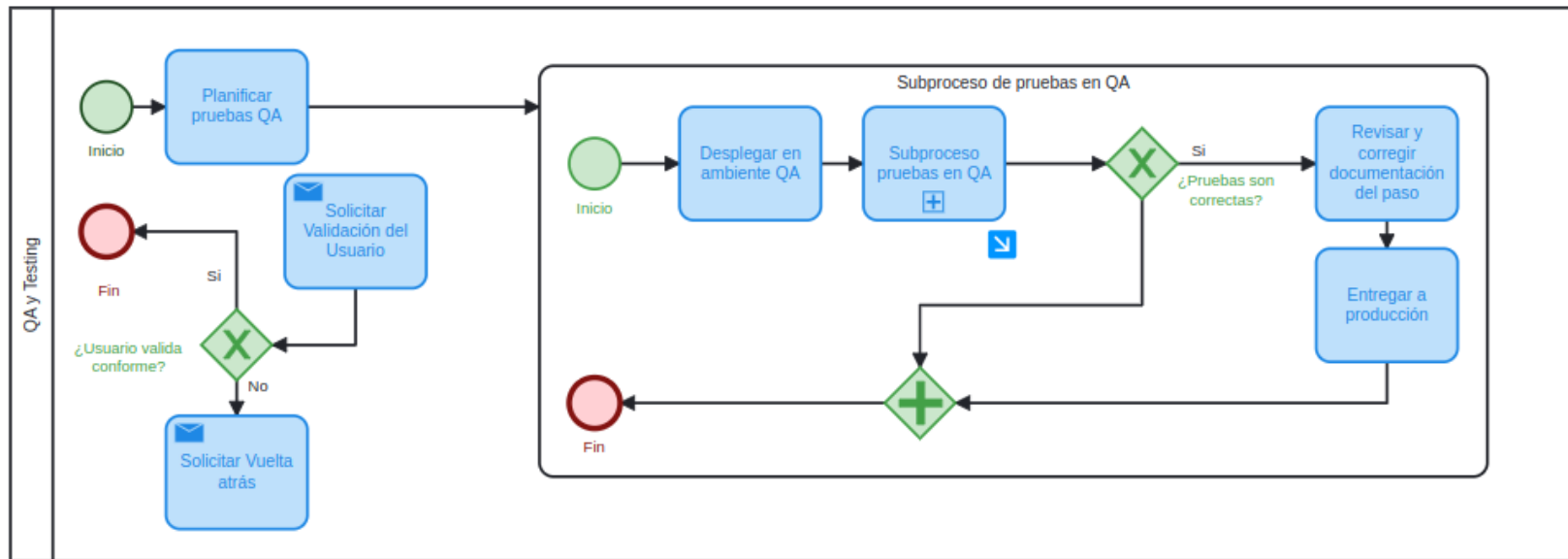


Figura 11. Actividades realizadas por el equipo de QA

La siguiente figura representa al subproceso “**Pruebas en QA**”.

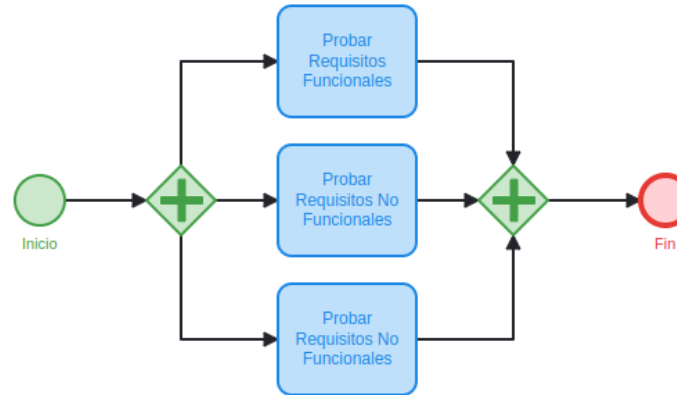


Figura 12. subprocesso “Pruebas en ambiente de Desarrollo”

QA y Testing	
Entradas	Salidas
- Historias de usuario entregadas por al área de Procesos y Negocios - Documento de paso a producción - Solicitud de pruebas en ambiente de QA desde Desarrollo	- Se entrega documento revisado a Operaciones TI - Se ejecutan pruebas (incluidas de seguridad), en caso de encontrar fallas se informa a Desarrollo para sus correcciones - Se envía a Operaciones TI para la ejecución de la aplicación a ambiente de producción - Se solicita validación usuaria - Se solicita vuelta atrás en caso de el usuario indique fallas en ambiente de producción

Operaciones TI

Actividad	Descripción	Actividad del modelo SAMM relacionada	Requisito PCI relacionado
3.4.31 Revisar documentación del paso	El despliegue del software en ambiente de producción debe ser ejecutado por un ingeniero de operaciones TI, quien revisa la documentación correspondiente en la plataforma de gestión de proyectos para verificar que los pasos a seguir se encuentren debidamente explicados. En caso contrario suspende la ejecución del paso hasta que la documentación sea corregida.	La actividad 3.4.29 se relaciona con la Práctica de Despliegue Seguro (SD1) del modelo OWASP SAMM, cuyo objetivo es que el procedimiento para el despliegue de las aplicaciones se encuentre correctamente documentado.	La actividad 3.4.29 ayuda a mejorar el cumplimiento de los siguientes requisitos de las normas PCI: 6. Desarrollar y mantener sistemas y aplicaciones seguros. 6.7 Asegúrese de que las políticas de seguridad y



Actividad	Descripción	Actividad del modelo SAMM relacionada	Requisito PCI relacionado
			los procedimientos operativos para desarrollar y mantener seguros los sistemas y las aplicaciones estén documentados, implementados y que sean de conocimiento para todas las partes afectadas.
3.4.32 Planificar ejecución del paso	El ingeniero de operaciones TI debe planificar el momento adecuado para la ejecución del despliegue, ya que éste puede afectar algunos servicios e incidir en las operaciones en línea y/o en el procesamiento de datos.		
3.4.33 Ejecutar paso	El ingeniero de operaciones TI tiene que proceder al despliegue automático del software por medio de la herramienta de despliegue continuo, validar la ejecución mediante la revisión de logs e informar a los interesados. Si el despliegue se ejecuta de manera correcta procede a dar aviso a los analistas de calidad para que informen al usuario y soliciten su validación en ambiente de producción. En caso de que durante el despliegue se produzca un error se procede a ejecutar la vuelta atrás del paso y éste es informado como fallido.	La actividad 3.4.31 se relaciona con la Práctica de Despliegue Seguro (SD1) del modelo OWASP SAMM, cuyo objetivo es que el procedimiento para el despliegue de las aplicaciones se encuentre correctamente documentado.	La actividad 3.4.31 ayuda a mejorar el cumplimiento de los siguientes requisitos de las normas PCI: 6. Desarrollar y mantener sistemas y aplicaciones seguros. 6.3 Desarrolle aplicaciones de software internas y externas (incluso acceso administrativo a aplicaciones basadas en web) de manera segura. 6.7 Asegúrese de que las políticas de seguridad y los procedimientos operativos para desarrollar y mantener seguros los sistemas y las aplicaciones estén documentados, implementados y que sean de conocimiento para todas las partes afectadas.
3.4.34 Ejecutar vuelta atrás	La ejecución de la vuelta atrás se produce en situaciones en las cuales se presentan errores durante el despliegue de los aplicativos en ambientes productivos o cuando el usuario informa errores en el funcionamiento de éstos. Esta es realizada por el ingeniero de TI responsable de la ejecución del paso.		
3.4.35 Informar vuelta atrás	La vuelta atrás debe ser informada a los interesados para que estén enterados de la situación y corrijan la causa de los errores presentados. Además, se debe verificar que el estado de los sistemas y/o procesos, que se encuentran en producción, no se haya visto afectado por la falla del paso.		

En el siguiente diagrama se presenta el flujo de actividades realizadas por el equipo de **Operaciones TI**

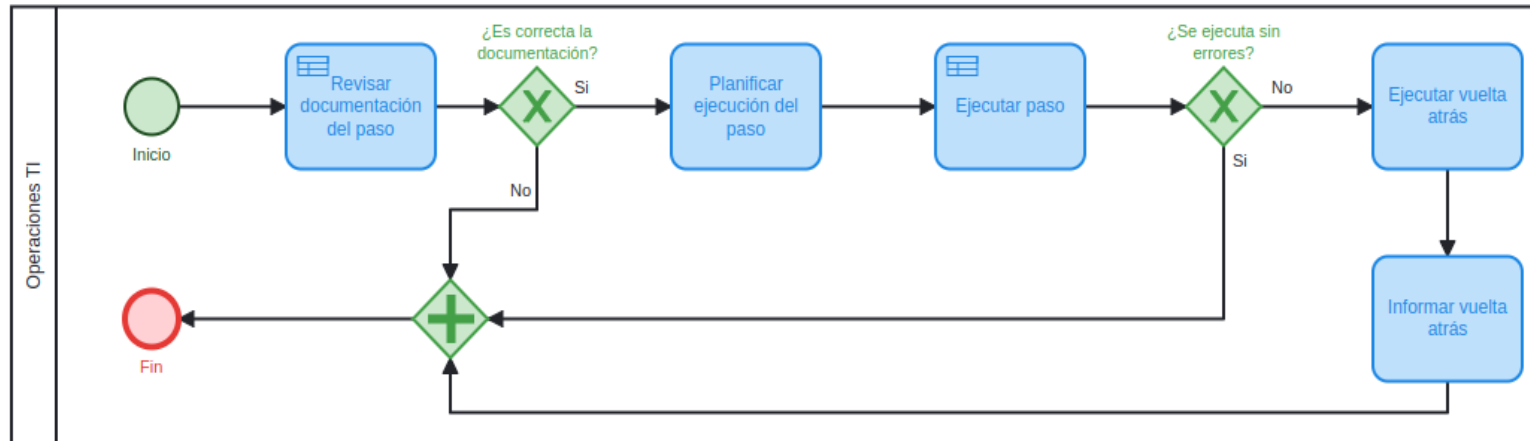


Figura 11. Actividades realizadas por el equipo de Operaciones TI

Operaciones TI	
Entradas	Salidas
- Documento de paso a producción - Solicitud de paso a producción - Solicitud de vuelta atrás si corresponde	- Se revisa documento de paso, se retorna a QA en caso de encontrar observaciones - Se da aviso de la instalación de la aplicación en ambiente de producción - Se ejecuta vuelta atrás en caso de error o solicitud de QA

Anexo 5 Soluciones y herramientas aplicables

En esta sección se presentan algunas herramientas y referencias que pueden servir de ayuda para realizar algunas de las actividades que forman parte de la metodología de desarrollo propuesta.

A.5.1 Requisitos de seguridad

La práctica de Requisitos de Seguridad indica que éstos se deben gestionar desde el inicio del proyecto tal y como se hace con los requisitos funcionales. Por ello, se deben utilizar las mismas herramientas con la que los equipos de proyecto gestionan los requisitos funcionales. Es así como la documentación de requisitos se puede llevar a cabo en alguna herramienta de gestión de proyectos.

A.5.1.1 Herramientas aplicables a la práctica requisitos de seguridad

Jira es una herramienta de gestión de proyectos desarrollada por Atlassian, sus inicios fueron en el año 2002 teniendo como objetivo hacer seguimiento a los inconvenientes que fueran surgiendo en el desarrollo de un software. Hoy en día ofrece una serie de paquetes (Jira Service Desk, Jira Core, Jira Software) cuyo objetivo se encuentra basado en optimizar el nivel productivo (productividad) de todos los miembros del equipo de trabajo, adicionalmente es posible utilizar Jira como un sistema de gestión de tareas.

Con el transcurso del tiempo, Jira ha buscado servir para cualquier tipo de gestión de proyectos, tales como el desarrollo de un sistema, el seguimiento de los errores que surgen durante el proceso de creación del software, gestión de contenidos, planificación de tareas de marketing, gestión de proyectos ágiles (Scrum), gestión de incidencias, entre otras.

URL: <https://www.atlassian.com/es/software/jira>

A.5.2 Evaluar el riesgo

Esta práctica trata sobre la evaluación del riesgo para cada una de las aplicaciones de la organización, así como el impacto que supondría para la ésta la materialización de dichos riesgos.

A.5.2.1 Herramientas aplicables a la práctica evaluar el riesgo

OWASP Risk Rating Methodology corresponde a una metodología que se puede utilizar para valorar los riesgos de las aplicaciones de forma cuantitativa y sistemática.

URL: https://wiki.owasp.org/index.php/OWASP_Risk_Rating_Methodology

OWASP Risk Assessment Calculator corresponde a una aplicación que permite realizar una evaluación de riesgos acorde a OWASP Risk Rating Methodology.

URL: <https://javierolmedo.github.io/OWASP-Calculator/>

A.5.3 Modelado de amenazas

El modelado de amenazas permite identificar vulnerabilidades y defectos durante la fase de diseño del software.

A.5.3.1 Herramientas aplicables a la práctica modelado de amenazas

Microsoft Threat Modeling Tool (STRIDE). Esta herramienta facilita el modelado de amenazas. Permite elaborar diagramas y dispone de un motor de reglas para autogenerar amenazas y propuestas de mitigación.

URL: <https://www.microsoft.com/en-us/securityengineering/sdl/threatmodeling>

A.5.4 Evaluar herramientas y tecnologías

Para evaluar y revisar las vulnerabilidades que pueden presentar los componentes de terceros que son utilizados para el desarrollo de software dentro de la organización, se pueden utilizar los siguientes sitios como fuente de información

A.5.4.1 Fuentes de información para evaluar herramientas y tecnologías

MAVENREPOSITORY. En este sitio se encuentran datos centralizados de las distintas bibliotecas para el desarrollo de aplicaciones java. Dentro de la información entregada en el sitio se encuentran las vulnerabilidades que se han detectado para estos componentes.

URL: <https://mvnrepository.com/>

CVE. El sitio de vulnerabilidades y exposiciones comunes (*Common Vulnerabilities and Exposures*) entrega una lista de información registrada sobre vulnerabilidades de seguridad conocidas, en la que cada referencia tiene un número de identificación CVE-ID, descripción de la vulnerabilidad, que versiones del software están afectadas, posible solución al fallo (si existe) o cómo mitigar la vulnerabilidad y referencias a publicaciones o entradas de foros o blogs donde se ha hecho pública la vulnerabilidad o se demuestra su explotación.

URL: <https://cve.mitre.org/>

NVD. El sitio de la base de datos nacional de vulnerabilidades (National Vulnerability Database) es el depósito del gobierno de EE. UU. de datos de gestión de vulnerabilidades basados en estándares representados mediante el protocolo de automatización de contenido de seguridad.

URL: <https://nvd.nist.gov/>

A.5.5 Construcción segura

Para ayudar a implementar las actividades de esta práctica, sí existen un buen número de herramientas que ayudan a dar cumplimiento a los controles de SAMM. Desde herramientas para automatizar los procesos de construcción hasta utilidades para verificar el cumplimiento de ciertos requisitos de seguridad, calidad, etc.

A.5.5.1 Herramientas aplicables a la práctica de construcción segura

Maven. Maven es una herramienta de software para la gestión y construcción de proyectos Java. Permite automatizar en gran medida el proceso de construcción del software, pero además es capaz de realizar otras tareas (a través de sus plugins) como pasar test unitarios y de integración, validar ciertos criterios de calidad, etc.

URL: <http://maven.apache.org/>

Jenkins. Jenkins es un servidor de automatización que sirve de gran ayuda para automatizar gran parte del proceso de construcción del software. Permite que la generación se realice de forma centralizada y es capaz de integrar la verificación, la obtención de métricas de calidad, la ejecución de tests y obtención del resultado, establecer “quality gates” para para el proceso de construcción ante ciertas condiciones, desplegar los artefactos construidos en un repositorio central, etc.

URL: <https://www.jenkins.io/>

Nexus Repository. Nexus es un repositorio de artefactos software donde se puede almacenar de forma centralizada las librerías y dependencias necesarias (y permitidas) para la construcción de productos software.

También podrá almacenar los productos construidos para diferentes entornos (testing, pre-producción, producción, etc.). Nexus ofrece soporte para repositorios de diversos tipos que almacena imágenes de Docker, paquetes npm, artefactos de Python y Go, etc.

URL: <https://www.sonatype.com/>

Sonarqube. Sonarqube es una plataforma de análisis estático de código (SAST)¹² que permite evaluar el código fuente antes de la construcción del producto final. Tras el análisis proporciona un conjunto de métricas de

¹² SAST es una herramienta de análisis estático que evalúa el código fuente de una aplicación para identificar vulnerabilidades de seguridad antes de que se implemente en un entorno de producción real.

calidad que se podrán utilizar posteriormente para mejorar la calidad del producto. También es capaz de detectar bugs y vulnerabilidades de seguridad en el código.

URL: <https://www.sonarqube.org/>

OWASP Dependency Check. Dependency Check es una herramienta que nos permite identificar las dependencias de nuestro proyecto y comprobar si tienen vulnerabilidades de seguridad conocidas. Se integra fácilmente con Maven a través del mecanismo de plugins.

URL: <https://owasp.org/www-project-dependency-check/>

A.5.6 Despliegue seguro

Para dar cumplimiento a algunos de los controles definidos por SAMM para esta práctica, sirven de ayuda algunas de las herramientas mencionadas anteriormente. Adicionalmente a éstas se puede incluir OWASP ZAP a la práctica de despliegue seguro.

A.5.6.1 Herramienta aplicables a la práctica de despliegue seguro

OWASP ZAP (Zed Attack Proxy). Es una herramienta DAST¹³ que permite escanear aplicaciones web para detectar vulnerabilidades de seguridad.

OWASP ZAP se interpone entre el navegador y la aplicación web, intercepta los mensajes, los inspecciona y puede modificar el contenido si es necesario. Permite detectar automáticamente vulnerabilidades básicas del OWASP Top 10 y otro tipo de vulnerabilidades.

URL: <https://www.zaproxy.org/>

A.5.7 Pruebas de seguridad

Para dar cumplimiento a los controles de SAMM establecidos para esta práctica se pueden utilizar las mismas herramientas mencionadas en puntos anteriores, así como la también mencionada OWASP ZAP.

A.5.7.1 Herramienta aplicables a la práctica pruebas de seguridad

OWASP Proactive Controls. Se trata de una especificación con una propuesta de controles específicos de seguridad. Se describen las categorías de controles más relevantes que deberían incluirse de forma incondicional en cualquier desarrollo de software.

URL: <https://owasp.org/www-project-proactive-controls/>

¹³ Las pruebas de seguridad de aplicaciones dinámicas (DAST), proporcionan una perspectiva externa de la aplicación antes del funcionamiento. Estas pruebas, también conocidas como "pruebas de caja negra", prueban las interfaces expuestas de una aplicación en ejecución en busca de vulnerabilidades y fallas, generalmente en aplicaciones web.

Anexo 6 Cuestionario de preguntas sobre la metodología propuesta y sus respuestas

6.1 Cuestionario de preguntas entregadas al equipo de trabajo

El cuestionario entregado al equipo de trabajo es el siguiente:

1. ¿Ha recibido información sobre normas de seguridad a las que se encuentra sujeta la organización?
2. ¿Ha recibido capacitación sobre metodologías de desarrollo de software seguro?
3. Califique su conocimiento sobre normas PCI antes de la realización de este trabajo.
4. Califique su conocimiento sobre normas PCI después de la realización de este trabajo.
5. Califique su conocimiento sobre modelos de madurez de aseguramiento de software antes de la realización de este trabajo.
6. Califique su conocimiento sobre modelos de madurez de aseguramiento de software después de la realización de este trabajo.
7. La calificación de riesgo es relevante para mejorar la seguridad del software.
8. El modelado de amenazas contribuye a mejorar la seguridad del software.
9. Los requisitos de seguridad se deben incorporar desde el inicio de un proyecto de software.
10. El análisis de diseño y arquitectura contribuye a la seguridad del software.
11. La creación de un inventario de componentes contribuye a mejorar el control sobre el proceso de desarrollo y la seguridad del software.
12. El uso de herramientas de revisión de calidad de código es relevante para la seguridad del software.
13. La actividad de revisión de código por pares para reducir los defectos y mejorar la seguridad del software.
14. Las pruebas de seguridad son tanto o más importantes que las funcionales.
15. Se deben ejecutar pruebas de seguridad durante la etapa de codificación del software.
16. El uso de herramientas para pruebas de penetración mejora la seguridad del software.
17. La metodología presentada es apropiada para ser utilizada en su trabajo.
18. La metodología presentada no aumenta considerablemente los tiempos de trabajo.
19. La metodología presentada ayuda a tener una visión más clara sobre el manejo de vulnerabilidades en el desarrollo de software.
20. La metodología presentada puede ser implementada en otros equipos de trabajo u organizaciones.

Las respuestas al cuestionario entregado al equipo de trabajo se encuentran en la sección “7.2 Respuestas a encuesta sobre la metodología propuesta”.

7.2 Respuestas a encuesta sobre la metodología propuesta

	Pregunta	Opción	Respuestas Encuestado 1	Respuestas Encuestado 2	Respuestas Encuestado 3	Respuestas Encuestado 4	Respuestas Encuestado 5	Respuestas Encuestado 6	Respuestas Encuestado 7	Respuestas Encuestado 8	Valores Promedio
1	¿Ha recibido información sobre normas de seguridad a las que se encuentra sujeta la organización?	1.Nunca 2.Raramente 3.Ocasionalmente 4.Frecuentemente 5.Muy Frecuentemente	2	2	3	2	3	4	2	3	2,625
2	¿Ha recibido capacitación sobre metodologías de desarrollo de software seguro?	1.Nunca 2.Raramente 3.Ocasionalmente 4.Frecuentemente 5.Muy Frecuentemente	2	2	2	3	3	3	3	2	2,5
3	Califique su conocimiento sobre normas PCI-DSS antes de la realización de este trabajo.	1.Nulo 2.Bajo 3.Medio 4.Medio alto 5.Avanzado	2	1	2	2	1	3	3	2	2
4	Califique su conocimiento sobre normas PCI-DSS después de la realización de este trabajo.	1.Nulo 2.Bajo 3.Medio 4.Medio alto 5.Avanzado	3	3	3	4	3	4	3	4	3,375
5	Califique su conocimiento sobre modelos de madurez de aseguramiento de software antes de la realización de este trabajo.	1.Nulo 2.Bajo 3.Medio 4.Medio alto 5.Avanzado	2	1	1	2	2	3	2	2	1,875
6	Califique su conocimiento sobre modelos de madurez de aseguramiento de software después de la realización de este trabajo.	1.Nulo 2.Bajo 3.Medio 4.Medio alto 5.Avanzado	3	3	4	3	3	4	4	4	3,5



	Pregunta	Opción	Respuestas Encuestado 1	Respuestas Encuestado 2	Respuestas Encuestado 3	Respuestas Encuestado 4	Respuestas Encuestado 5	Respuestas Encuestado 6	Respuestas Encuestado 7	Respuestas Encuestado 8	Valores Promedio
7	La calificación de riesgo es relevante para mejorar la seguridad del software.	1.Totalmente en desacuerdo 2.Desacuerdo 3.Ni de acuerdo, ni en desacuerdo 4.De acuerdo 5.Totalmente de acuerdo	4	4	5	4	4	5	4	4	4,25
8	El modelado de amenazas contribuye a mejorar la seguridad del software.	1.Totalmente en desacuerdo 2.Desacuerdo 3.Ni de acuerdo, ni en desacuerdo 4.De acuerdo 5.Totalmente de acuerdo	5	5	5	5	4	4	5	5	4,75
9	Los requisitos de seguridad se deben incorporar desde el inicio de un proyecto de software.	1.Totalmente en desacuerdo 2.Desacuerdo 3.Ni de acuerdo, ni en desacuerdo 4.De acuerdo 5.Totalmente de acuerdo	5	5	5	5	5	5	5	5	5
10	El análisis de diseño y arquitectura contribuye a la seguridad del software.	1.Totalmente en desacuerdo 2.Desacuerdo 3.Ni de acuerdo, ni en desacuerdo 4.De acuerdo 5.Totalmente de acuerdo	5	5	5	5	5	5	5	5	5
11	La creación de un inventario de componentes contribuye a mejorar el control sobre el proceso de desarrollo y la seguridad del software.	1.Totalmente en desacuerdo 2.Desacuerdo 3.Ni de acuerdo, ni en desacuerdo 4.De acuerdo 5.Totalmente de acuerdo	4	4	5	5	4	4	4	5	4,375



	Pregunta	Opción	Respuestas Encuestado 1	Respuestas Encuestado 2	Respuestas Encuestado 3	Respuestas Encuestado 4	Respuestas Encuestado 5	Respuestas Encuestado 6	Respuestas Encuestado 7	Respuestas Encuestado 8	Valores Promedio
12	El uso de herramientas de revisión de calidad de código es relevante para la seguridad del software.	1.Totalmente en desacuerdo 2.Desacuerdo 3.Ni de acuerdo, ni en desacuerdo 4.De acuerdo 5.Totalmente de acuerdo	5	5	5	5	5	5	5	5	5
13	La actividad de revisión de código por pares para reducir los defectos y mejorar la seguridad del software.	1.Totalmente en desacuerdo 2.Desacuerdo 3.Ni de acuerdo, ni en desacuerdo 4.De acuerdo 5.Totalmente de acuerdo	5	5	4	4	5	4	4	4	4,375
14	Las pruebas de seguridad son tanto o más importantes que las funcionales.	1.Totalmente en desacuerdo 2.Desacuerdo 3.Ni de acuerdo, ni en desacuerdo 4.De acuerdo 5.Totalmente de acuerdo	5	5	5	4	4	5	4	4	4,5
15	Se deben ejecutar pruebas de seguridad durante la etapa de codificación del software.	1.Totalmente en desacuerdo 2.Desacuerdo 3.Ni de acuerdo, ni en desacuerdo 4.De acuerdo 5.Totalmente de acuerdo	4	4	3	3	4	4	4	4	3,75
16	El uso de herramientas para pruebas de penetración mejora la seguridad del software.	1.Totalmente en desacuerdo 2.Desacuerdo 3.Ni de acuerdo, ni en desacuerdo 4.De acuerdo 5.Totalmente de acuerdo	4	4	4	5	4	4	5	5	4,375
17	La metodología presentada es apropiada para ser utilizada en su trabajo.	1.Totalmente en desacuerdo 2.Desacuerdo 3.Ni de acuerdo, ni en desacuerdo 4.De acuerdo 5.Totalmente de acuerdo	4	4	4	4	5	4	4	4	4,125



	Pregunta	Opción	Respuestas Encuestado 1	Respuestas Encuestado 2	Respuestas Encuestado 3	Respuestas Encuestado 4	Respuestas Encuestado 5	Respuestas Encuestado 6	Respuestas Encuestado 7	Respuestas Encuestado 8	Valores Promedio
18	La metodología presentada no aumenta considerablemente los tiempos de trabajo.	1.Totalmente en desacuerdo 2.Desacuerdo 3.Ni de acuerdo, ni en desacuerdo 4.De acuerdo 5.Totalmente de acuerdo	4	4	4	4	5	4	3	4	4
19	La metodología presentada ayuda a tener una visión más clara sobre el manejo de vulnerabilidades en el desarrollo de software.	1.Totalmente en desacuerdo 2.Desacuerdo 3.Ni de acuerdo, ni en desacuerdo 4.De acuerdo 5.Totalmente de acuerdo	4	4	4	5	4	5	5	4	4,375
20	La metodología presentada puede ser implementada en otros equipos de trabajo u organizaciones.	1.Totalmente en desacuerdo 2.Desacuerdo 3.Ni de acuerdo, ni en desacuerdo 4.De acuerdo 5.Totalmente de acuerdo	4	4	5	4	3	4	4	5	4,125

Participantes

Los participantes en este trabajo de investigación que aportaron con sus respuestas, observaciones y conocimiento son los siguientes:

Nombre	Cargo	Correo
Claudia Vázquez	Scrum/Líder Técnico	claudia.vasquez@multicaja.cl
Karin Hernández	Desarrolladora	karin.hernandez@multicaja.cl
Alejandra Vidal	Desarrolladora	alejandra.vidal@multicaja.cl
Elizabeth Canales	Desarrolladora	elizabeth.canales@multicaja.cl
Patricia Moema	Analista QA	patricia.moema@multicaja.cl
Daniel Domínguez	Desarrollador	daniel.dominguez@multicaja.cl
Rafael Rozas	Desarrollador	rafael.rozas@multicaja.cl
Alejandro Pacheco	Analista líder QA	alejandro.pacheco@multicaja.cl
Manuel Cortés	Desarrollador	mamuel.cortes@multicaja.cl